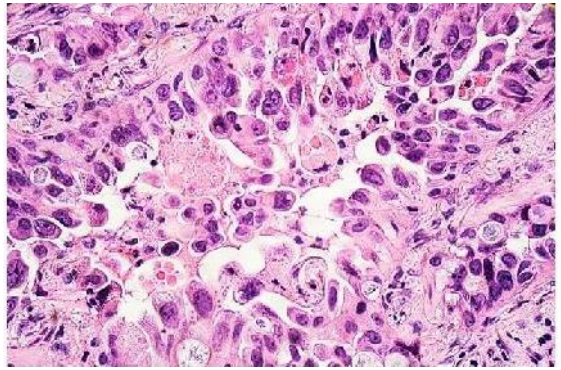
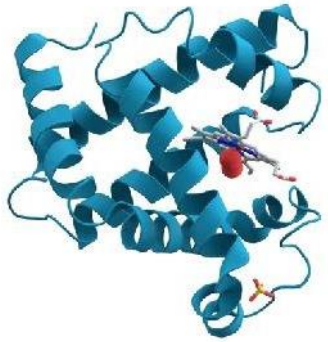
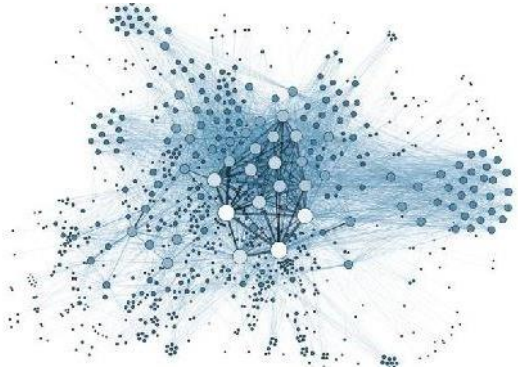
HARVARD MEDICAL SCHOOL

Core for Computational Biomedicine (CCB)



Website: https://ccb.hms.harvard.edu

Email: ccb_help@hms.harvard.edu

AI Seminar & Workshop Series

SPRING 2026

02/11	Large Language Models at HMS: Tools, Access, and Use Cases
03/11	Prompt Engineering
04/01	AI-Accelerated Literature Review and Scientific Writing
04/14	AI-Powered Vibe Coding
04/29	Posit AI Tools @ Harvard: AI integration in R and Python

FALL 2026

09/24	Agentic AI for Biomedical Data Science
10/08	Building AI Co-Scientists with ToolUniverse
11/05	**Agentic AI with Local Models**

Today · Countway Library, Room L1-032 and Zoom · 2:00–4:00 PM



Recordings, slides, and supplemental materials: https://ccb.hms.harvard.edu/

Core for Computational Biomedicine (CCB)

HARVARD MEDICAL SCHOOL

- **Free consulting and analytic support**
  - Computational Biology
  - Data science & AI
- **Collaboration on research projects**
  - Timeline: weeks to months
- **Office hours with experts**
- **Support for**
  - Generative AI and LLMs
  - Programming (R, Python, …)
  - Biostatistics & Bioinformatics
  - High-performance compute infrastructure
  - Data analysis pipelines
  - Handle large data sets
  - Prototype models

Need help?



CCB Office Hours

In person on Wednesdays,
or on Zoom any day

Contact us if you need help with your project!

Website: https://ccb.hms.harvard.edu/

Email: ccb_help@hms.harvard.edu

Agenda

1 Why local inference?

2 Choosing a model

3 Talking to a model

4 Walkthrough

5 Using models well

6 Beyond the workshop

For information about data security, contact the Office of Research Administration https://researchadmin.hms.harvard.edu/

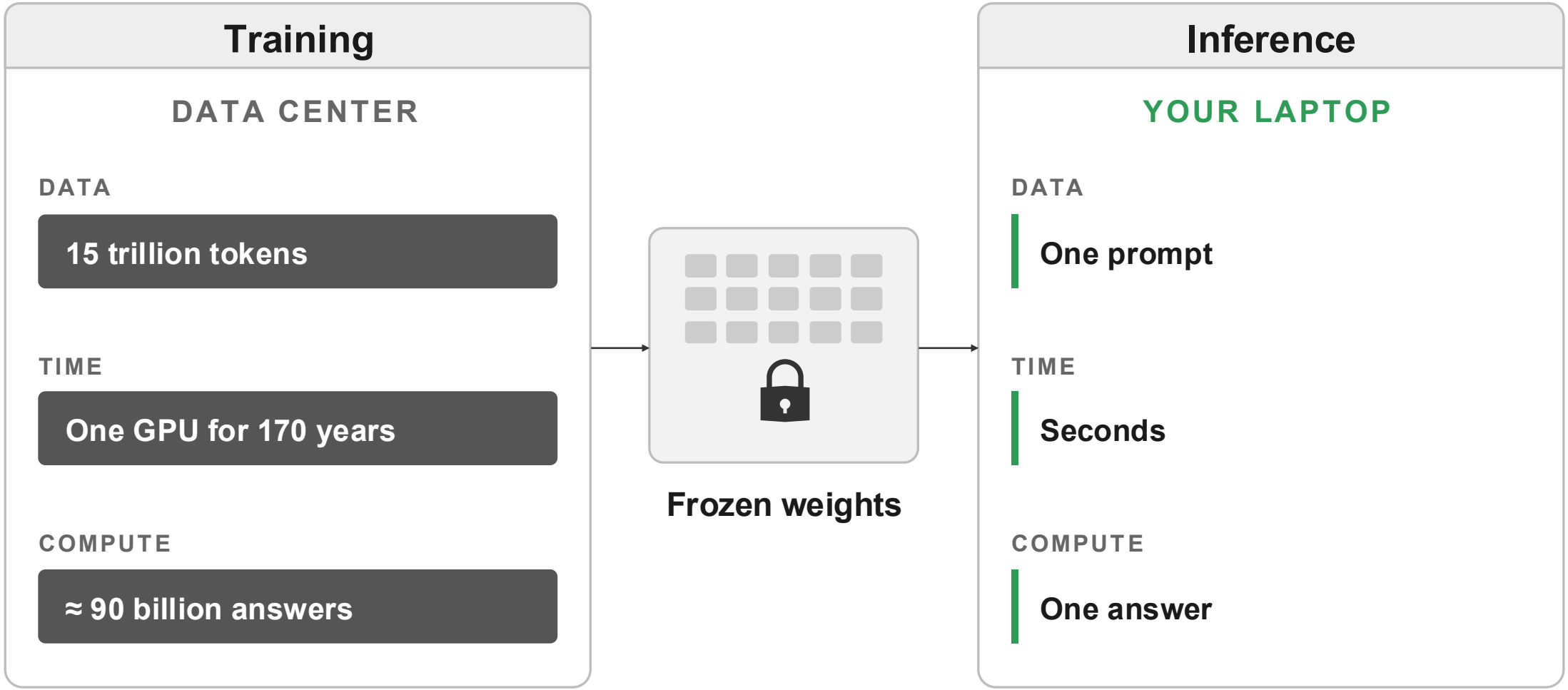


HARVARD MEDICAL SCHOOL | BLAVATNIK INSTITUTE BIOMEDICAL INFORMATICS

Why Local Inference?

Grey Kuling

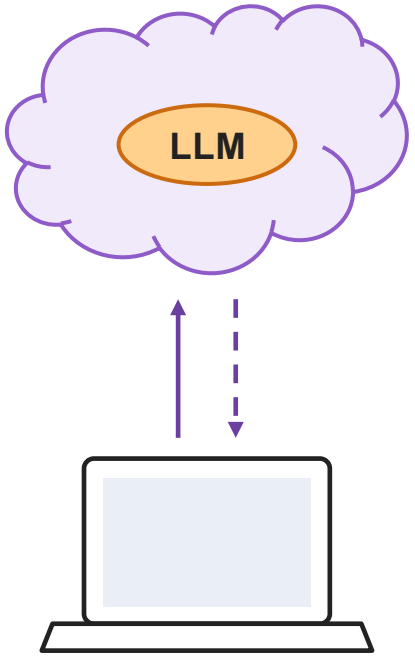
Training versus inference



Example: Llama 3.1 8B. Source: Meta, Llama 3.1 model card (15T tokens, 1.46M H100 GPU-hours). Answers = estimated compute, 500-token reply.

What is local inference?

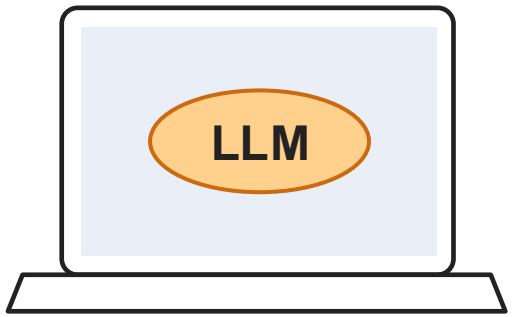
Cloud inference



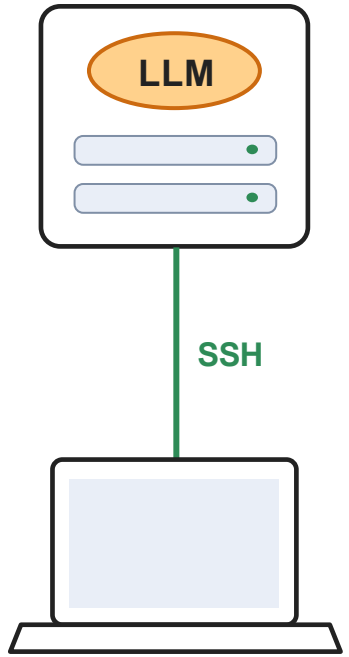
✗ Data is transmitted to a company's servers

Local inference

On your laptop



On HMS hardware



✓ Data stays on your machine / within the HMS network

Why choose a local model?

LOCAL INFERENCE AFFORDS:



Working offline

Work on an airplane



Privacy controls

You manage data egress



Version control

Save model version and weights

SMALLER MODELS AFFORD:



Cost efficiency

No per-token payments



Energy efficiency

Smaller models use less power



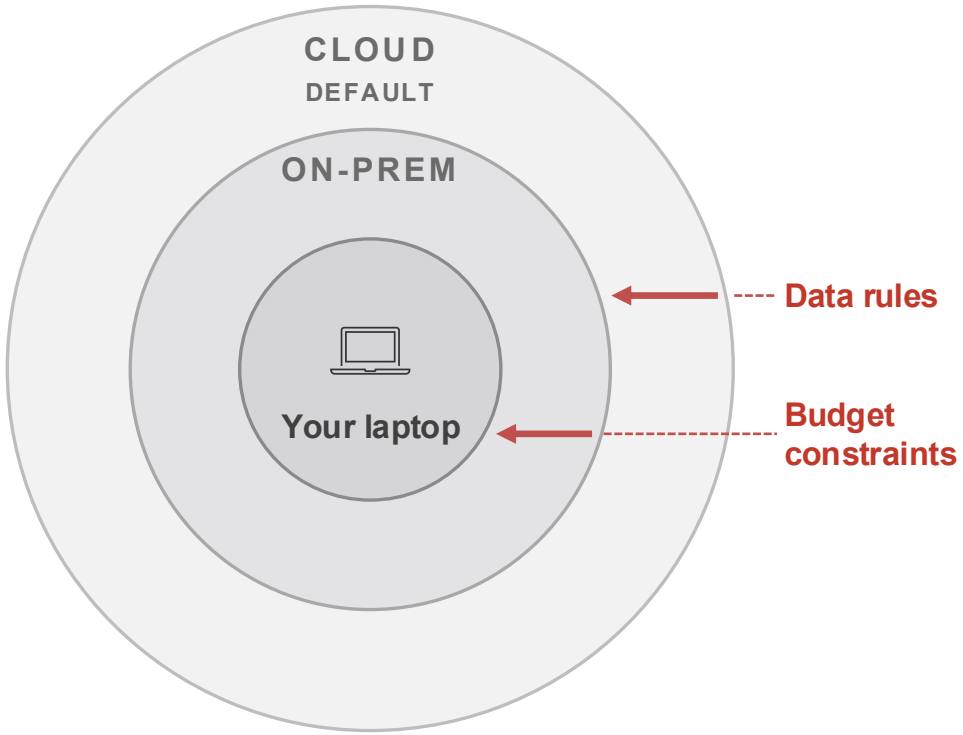
Speed

Smaller models can be faster

As models get better, we can begin to shift our framing…

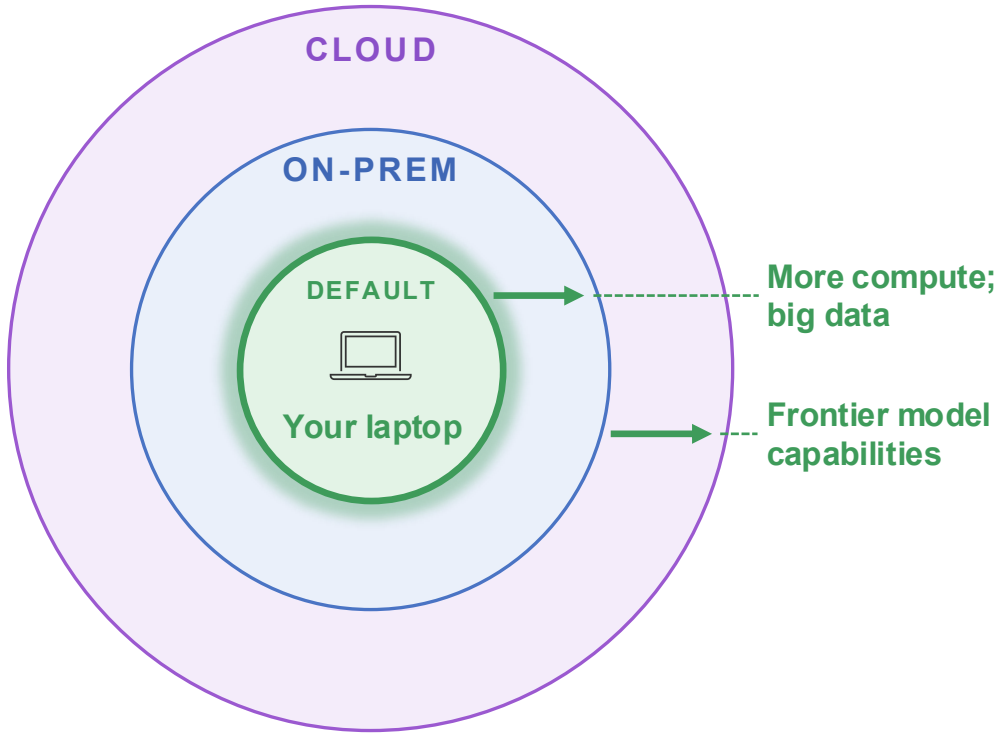
The usual framing

Moving local feels like our world is shrinking



A new framing

Local is our home base; we venture out as needed



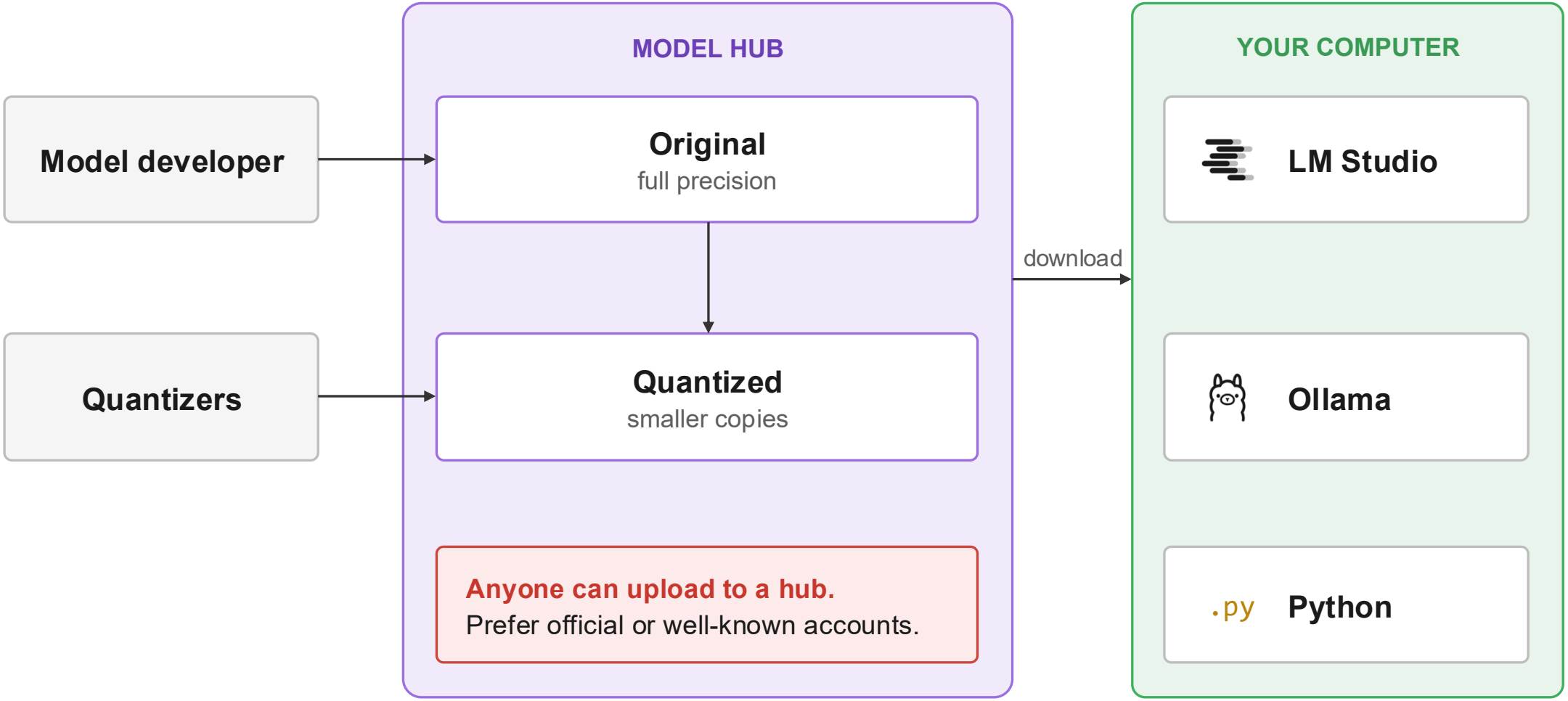


HARVARD MEDICAL SCHOOL | BLAVATNIK INSTITUTE BIOMEDICAL INFORMATICS

Choosing A Model

Grey Kuling

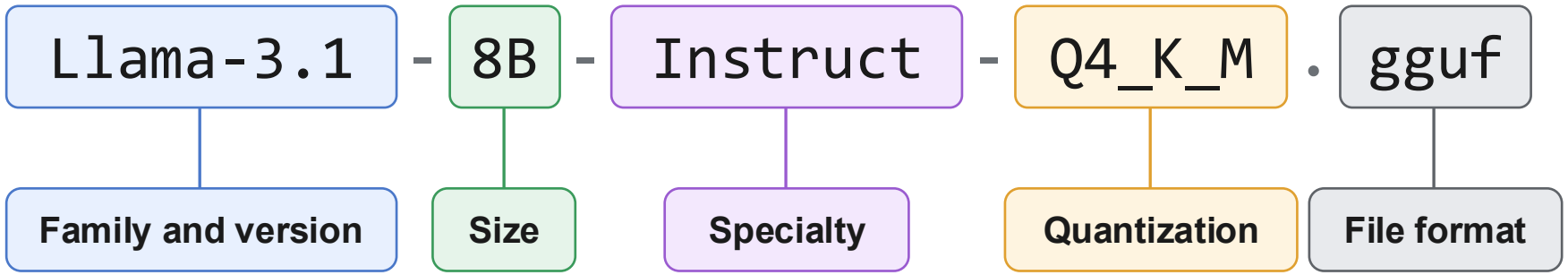
How a model reaches your computer



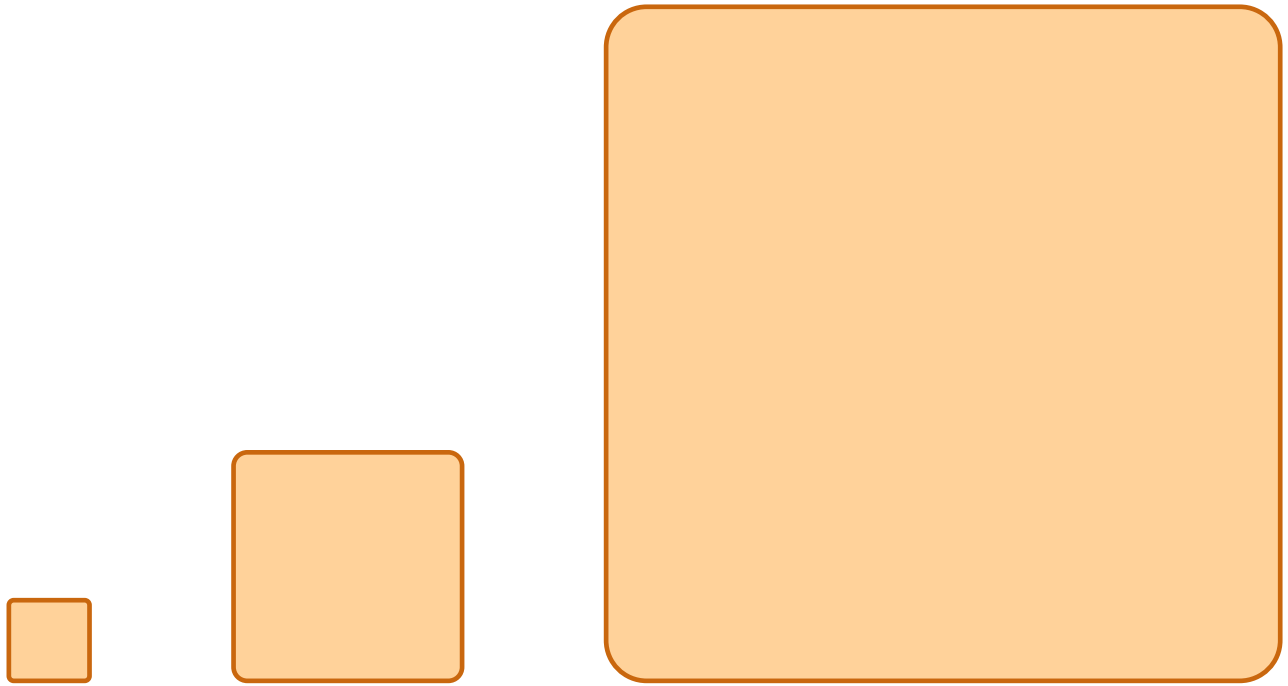
Model Hubs

SOURCE	BEST FOR
Hugging Face Hub	Nearly every open model, in every format
Ollama Library	Ready-to-run models for Ollama: `ollama pull qwen3`
LM Studio	Searches Hugging Face and flags what fits your memory
NVIDIA NGC	Models tuned for NVIDIA GPU servers
BioImage Model Zoo	Expert microscopy models, not LLMs

Decoding model names in the marketplace



Model size: parameters and memory



1B
≈ 2 GB

8B
≈ 16 GB

70B
≈ 140 GB

Parameters

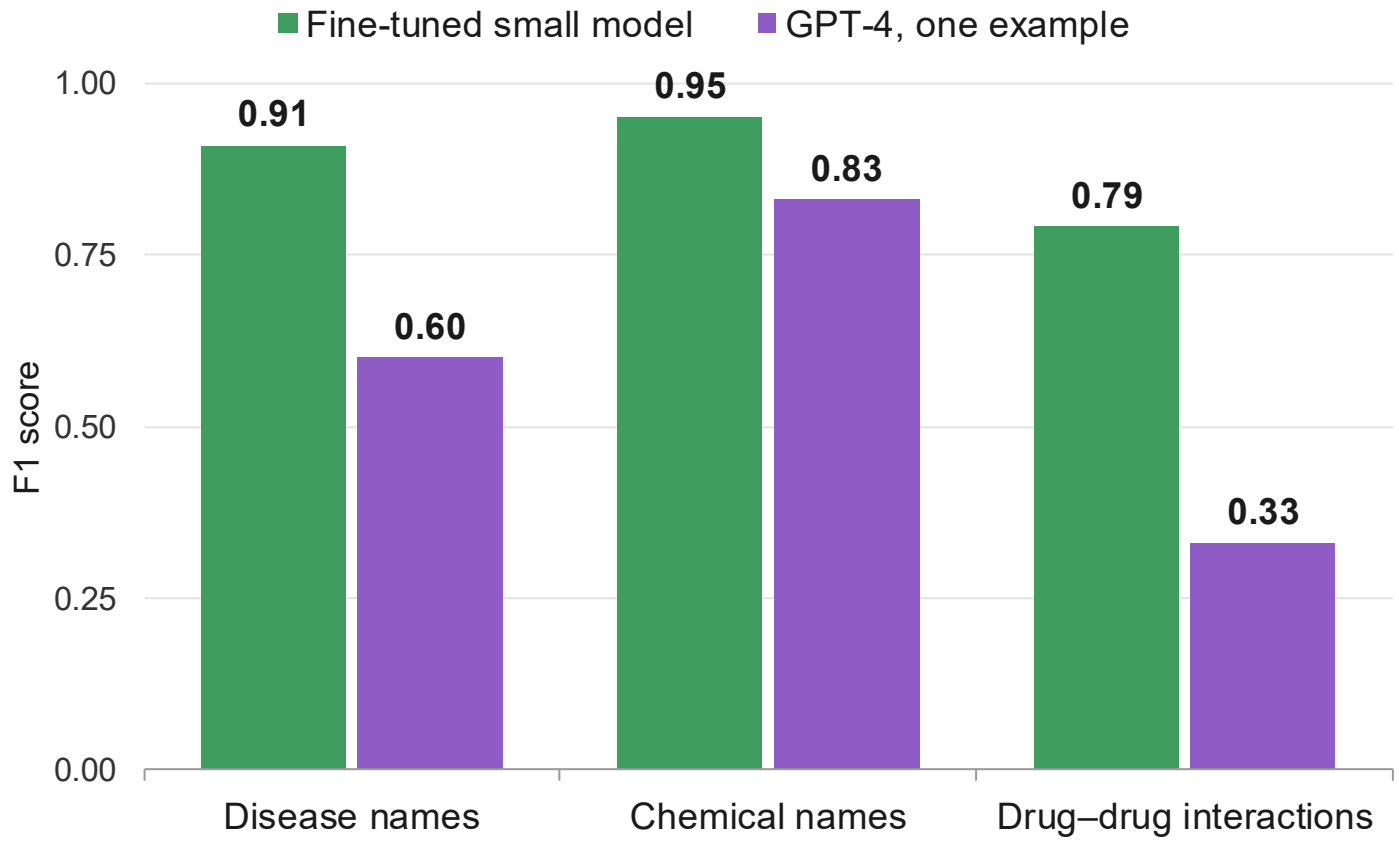
the learned weights
inside a model

Memory at 16-bit

parameters × 2 bytes

Bigger is not always better

For pulling facts out of biomedical text, small fine-tuned models beat GPT-4.



Small specialist models (110M)

PubMedBERT and BioBERT: about 110M parameters, fine-tuned for each specific medical information task

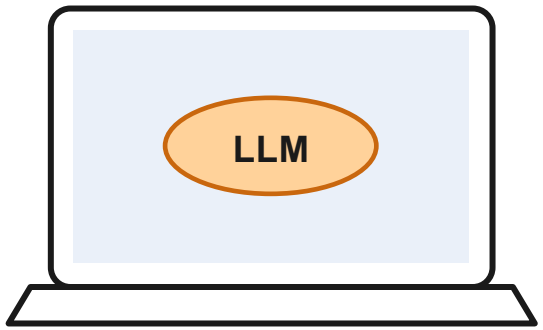
Large language model (1.5T)

GPT-4 (~1.5 trillion parameters) was stronger on reasoning, such as medical question answering, not *facts*

Chen et al. (2025), Nature Communications
doi:10.1038/s41467-025-56989-2

Tradeoffs of small models

Small model on your laptop



Pros

- ✓ Privacy
- ✓ Low cost

Cons

- ! Speed
- ! Accuracy
- ! Tool use
- ! Ease of prompting

Specialties and modalities

Specialty

Base

Continues the text it is given. Not built to follow instructions.

`-Base` `-pt`

Instruct

Follows instructions and chats. Use this for most tasks.

`-Instruct` `-it` `-Chat`

Thinking

Writes out its reasoning first. Better on hard problems, but slower.

`-Thinking` `-Reasoning` `-R1`

Modality

`-Coder`
Writing and editing code

`-VL -Vision`
Reads images as well as text

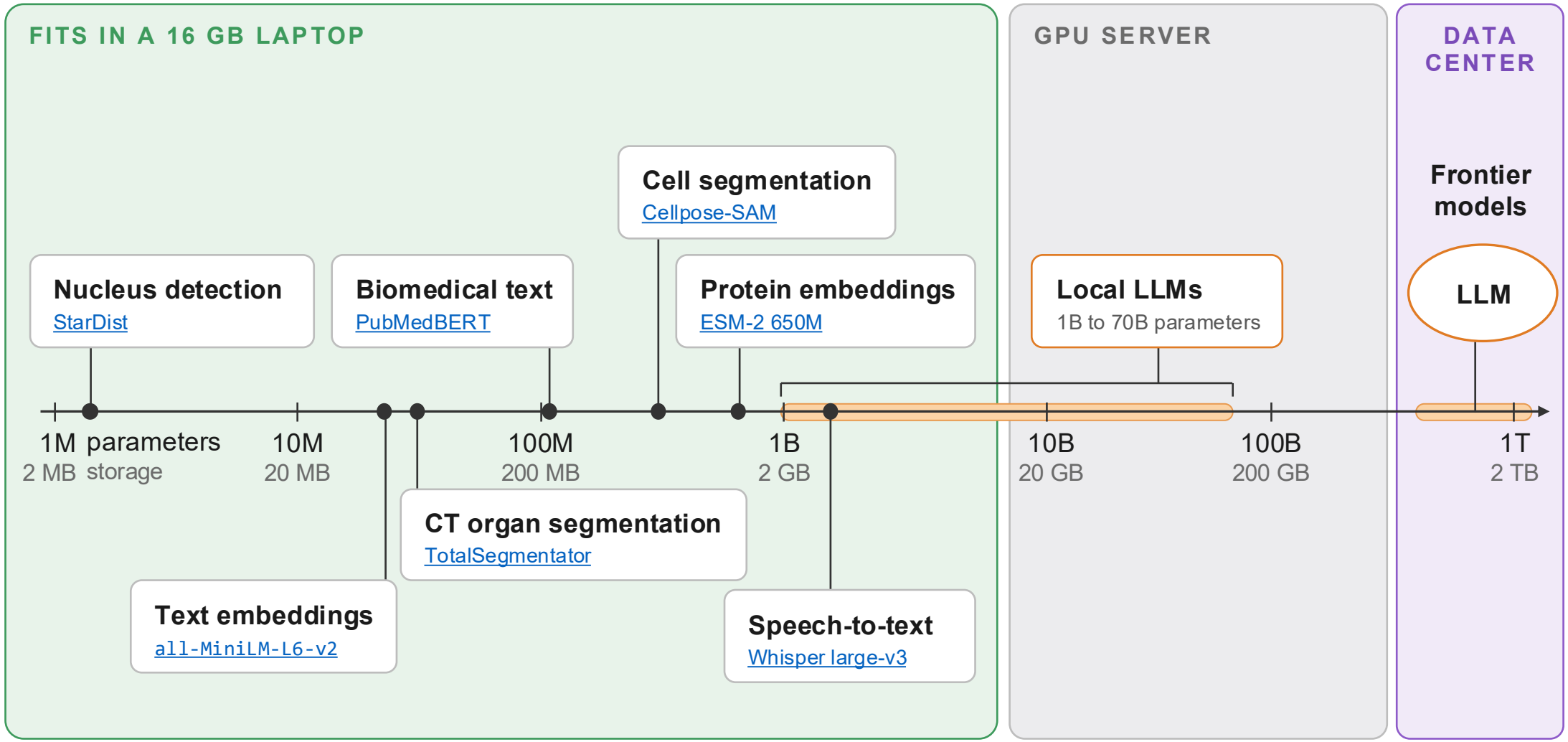
`-Math`
Mathematical problem solving

`-Embedding`
Text to vectors; does not chat

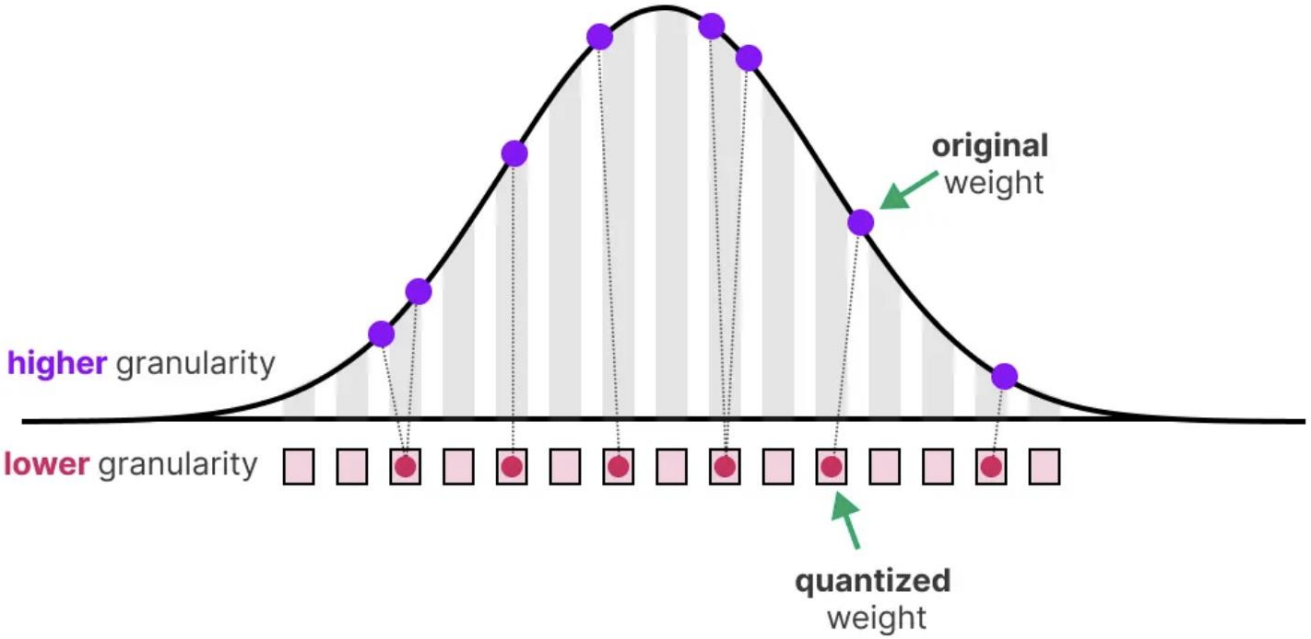
`med`
Further trained on medical data

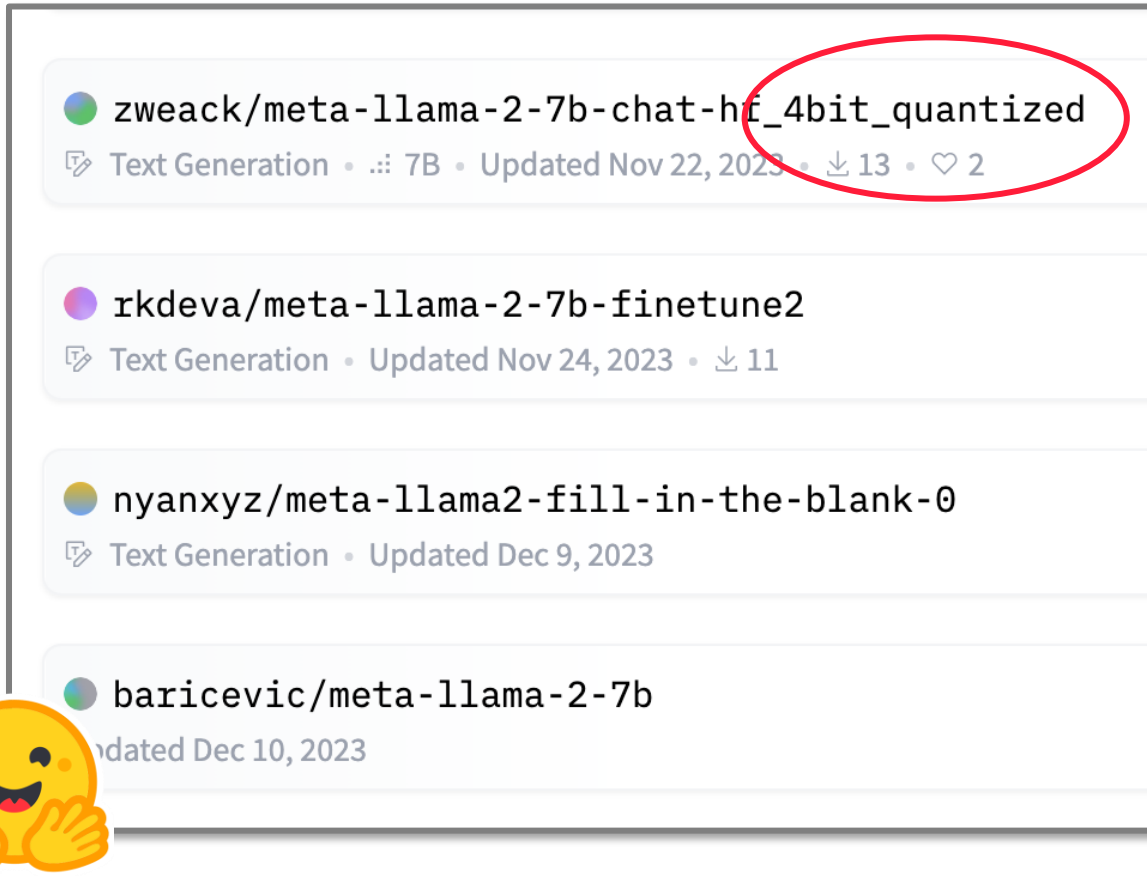
`-Guard`
Labels text as safe or unsafe

Specialized models for specialized tasks



Quantization: Condensing from floating points to integers

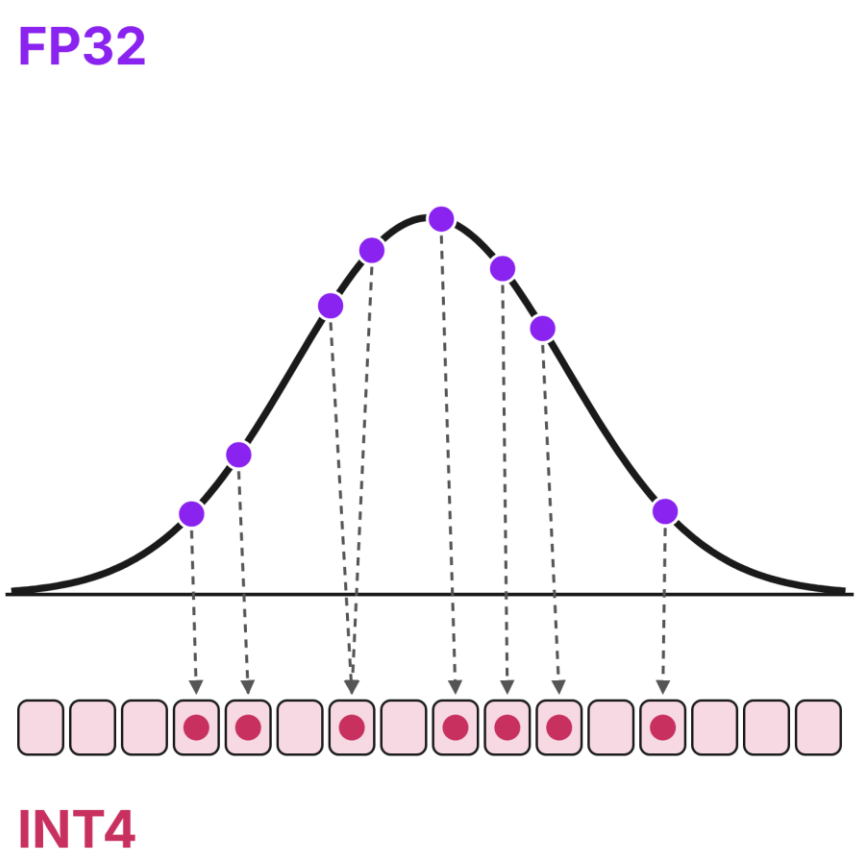




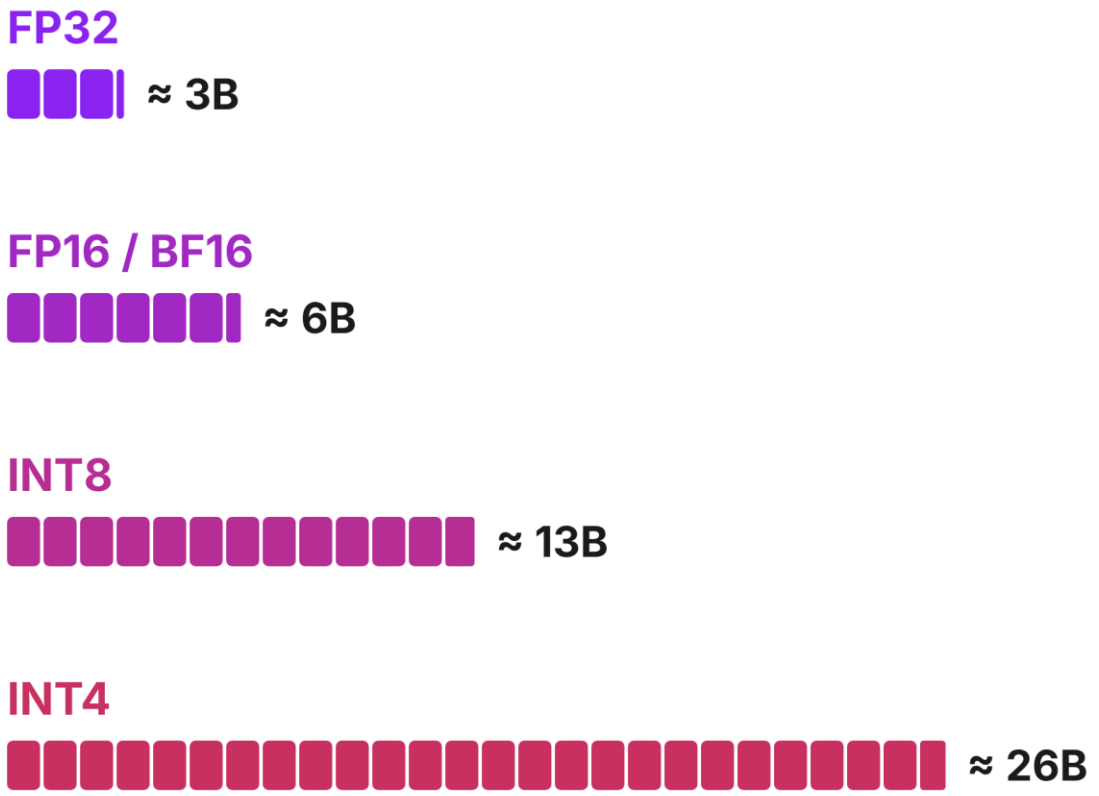
"A Visual Guide to Quantization" Maarten Grootendorst

Quantization allows higher-parameter models to fit in smaller spaces

Rounding weights to fewer levels



Largest model that fits in 16 GB



CPU vs GPU



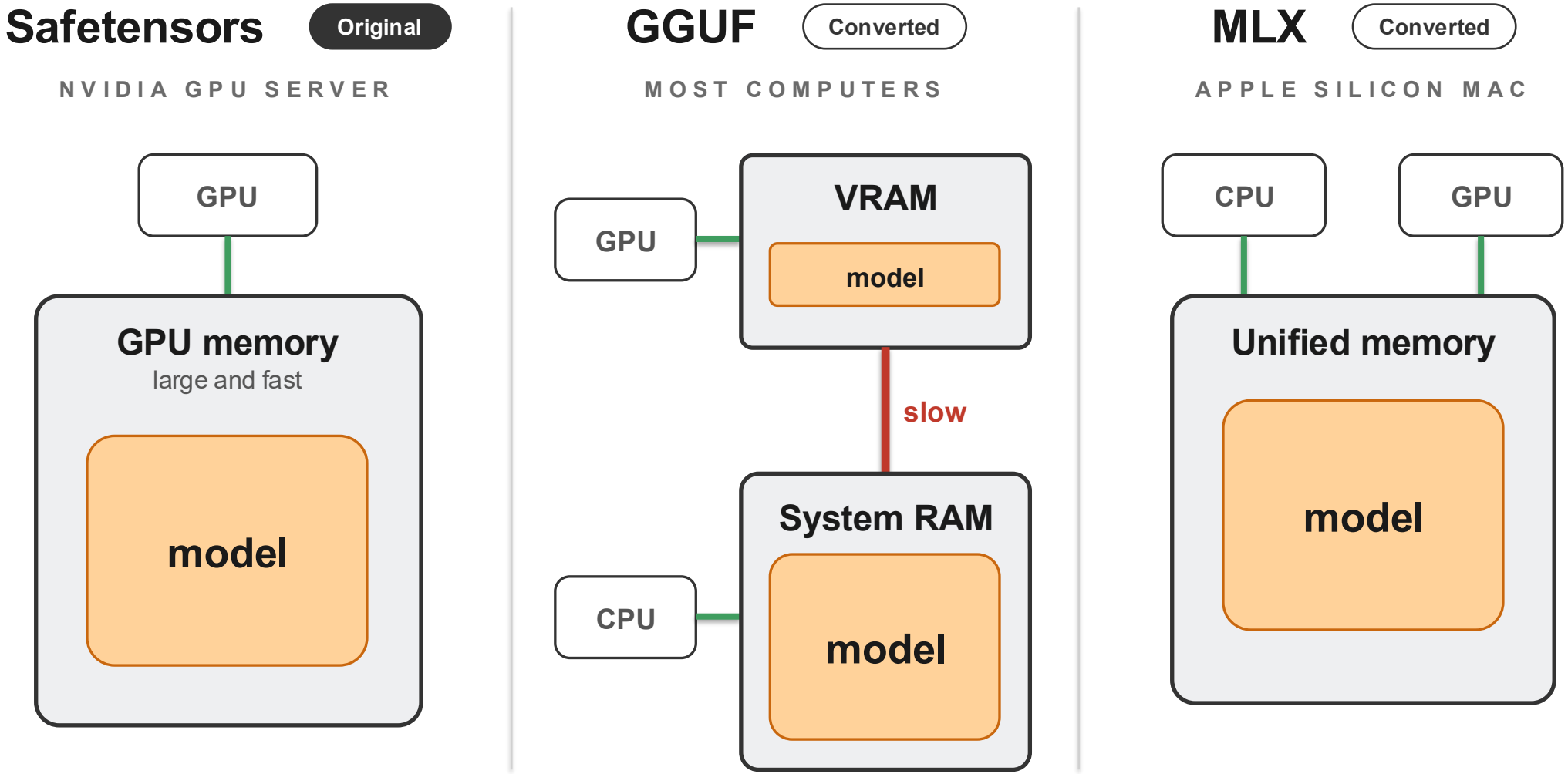


CPUs are very good at moving things quickly, one thing at a time

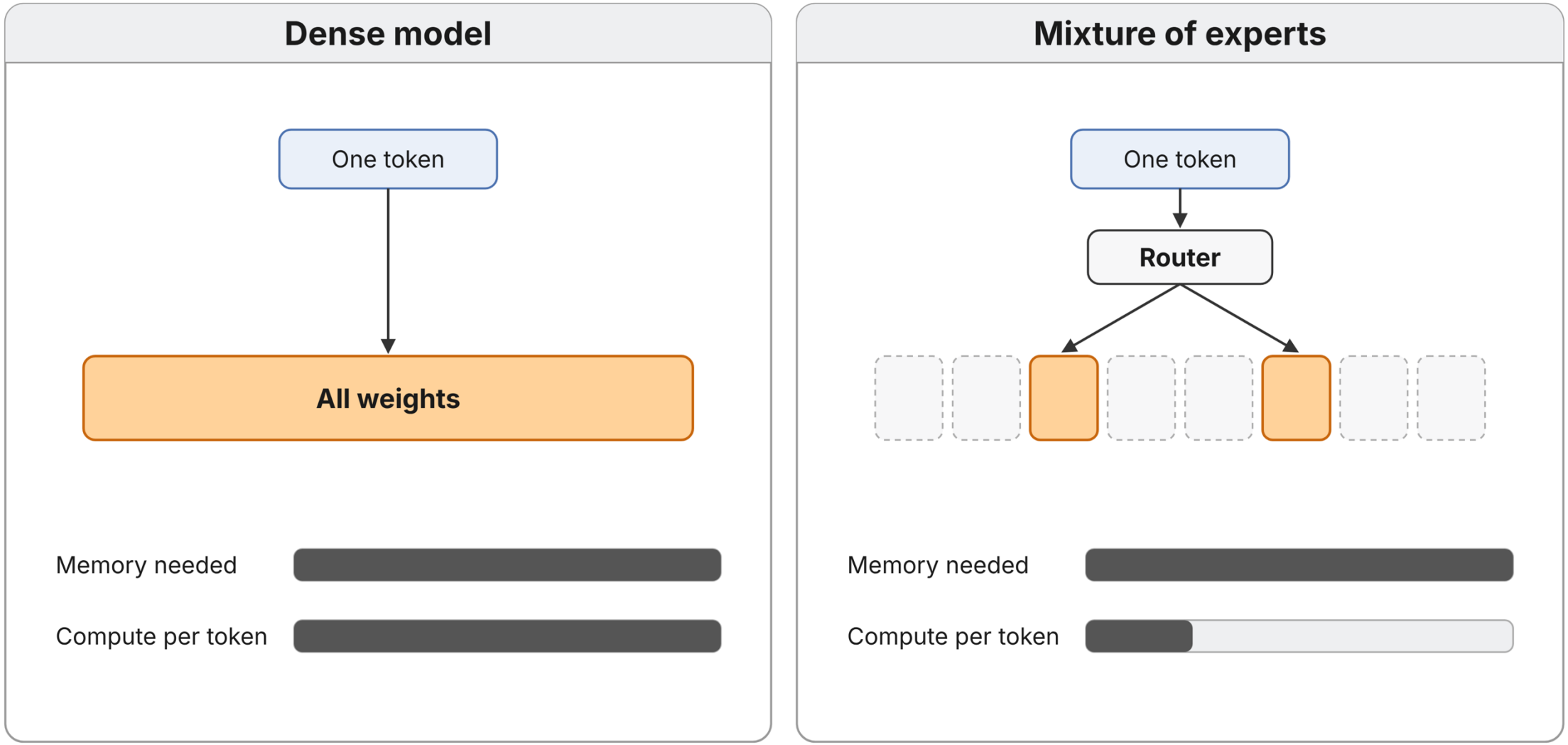
GPUs are very good at moving a lot of the same thing all at once

Typical CPU architecture vs a typical GPU architecture

Certain file formats are optimized for certain hardware

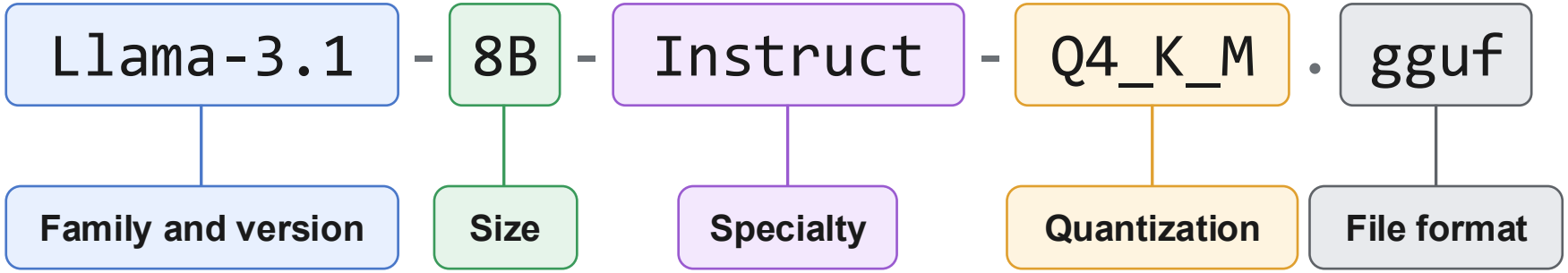


Mixture-of-Experts (MoE) models can reduce compute-per-token

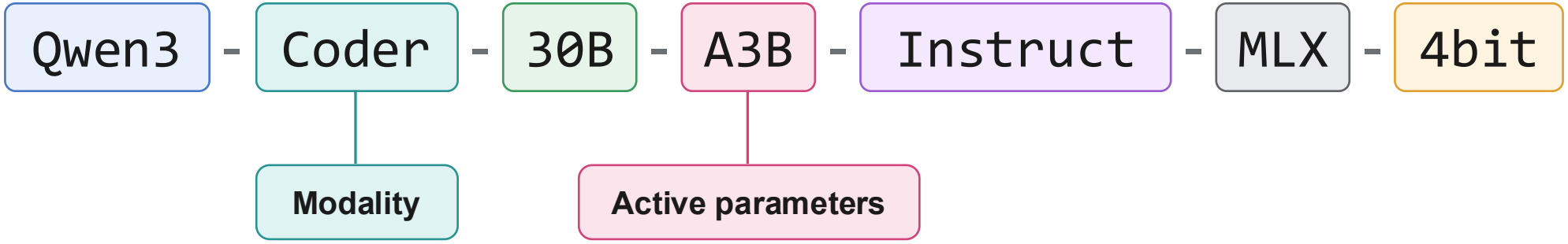


Decoding model names in the marketplace

A dense model



A mixture-of-experts model



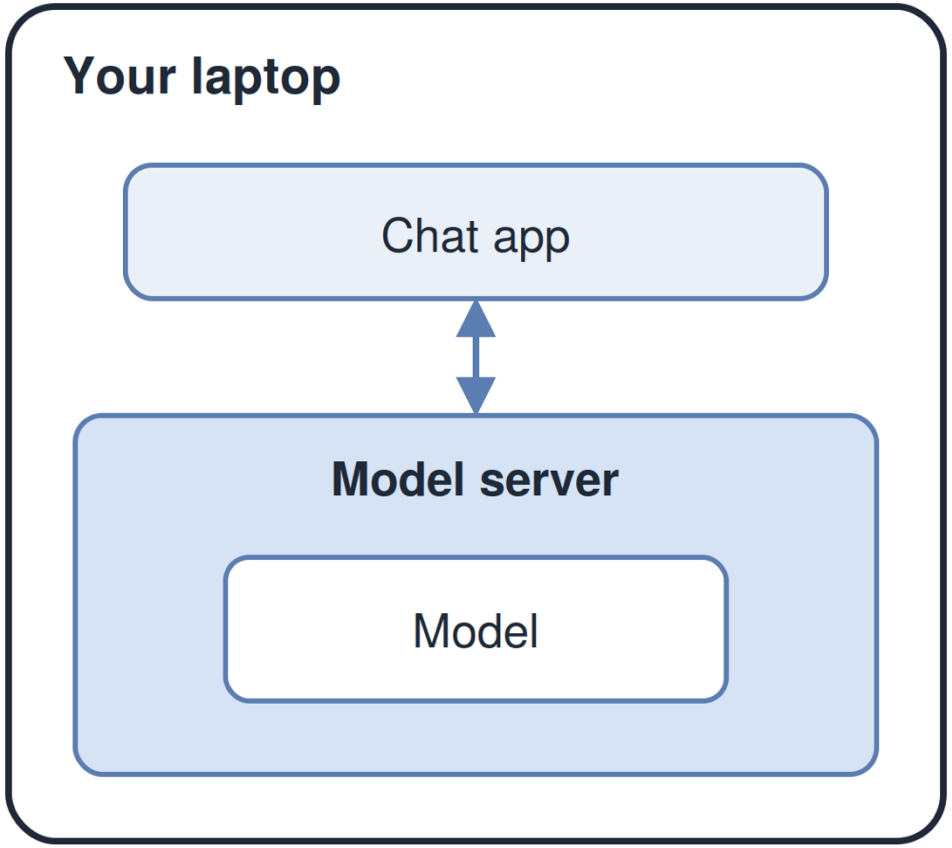


HARVARD MEDICAL SCHOOL | BLAVATNIK INSTITUTE BIOMEDICAL INFORMATICS

Talking to a Model

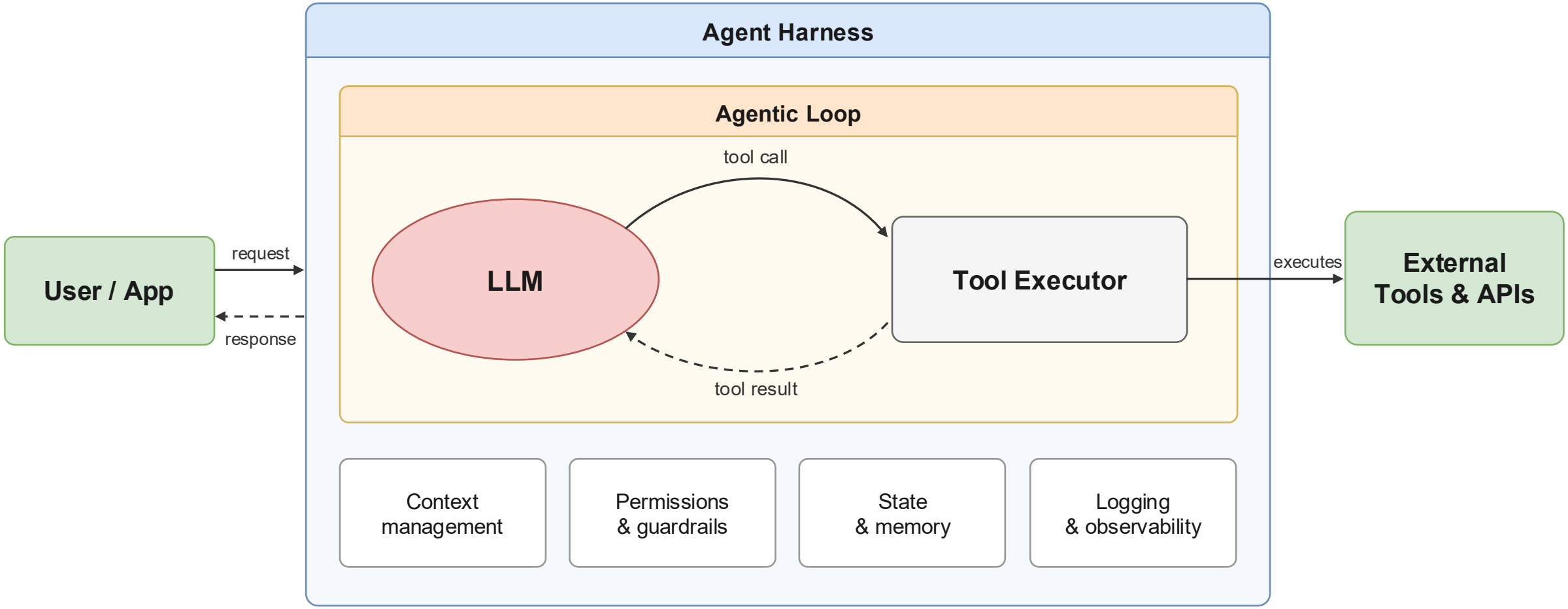
Jane Adams

What is a model server?



Server	Runs on
Ollama	
LM Studio	
Jan	
MLX-LM	
oMLX	

What is a harness?



"Agent harness is just system design with a new name", Akshay Kokane

Local inference software types

Servers with chat built in

Run the model and chat in one app

- LM Studio
- gpt4all
- Ollama
- Jan

Chat interfaces

Connect to a model server you run

- Open WebUI
- Anything LLM
- LibreChat
- LobeHub
- CHERRY STUDIO

Coding harnesses

Read, edit, and run code for you

- opencode
- CLAUDE CODE
- Codex
- Cline
- goose
- Kilo Code

What is a tool?

"What time is it?"

```
def get_time():
    return datetime.now()
```

A tool is a function plus a description.

Inside, it can call other **functions**, **APIs**, **write files**, or even ask **another LLM**.

"Analyze this gene: BRCA2"

```
def run_analysis(gene):
```

API call: fetch gene variants

Local LLM: summarize findings

Write file: save report.md

```
return result
```

shown as

What the model sees:

NAME

`run_analysis`

DESCRIPTION

Analyze a gene and save a summary report

INPUTS

gene (text, e.g. BRCA2)

Not all models use the same tool-calling syntax

OpenAI style

```
"tool_calls": [{
  "function": {
    "name": "get_expression",
    "arguments":
      {"gene": "BRCA2"}
  }
}]
```

GPT (via API)

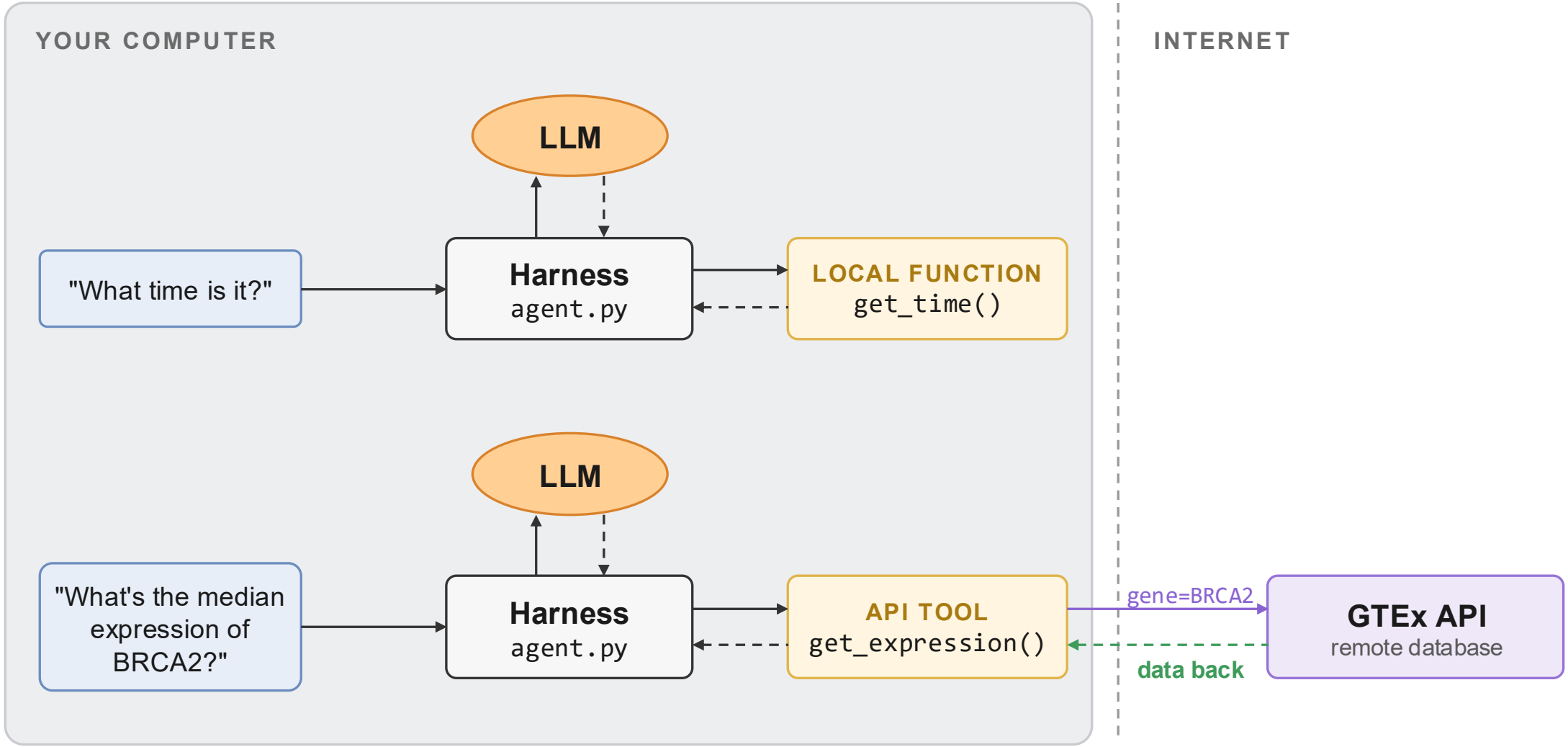
XML style

```
<function=get_expression>
  <parameter=gene>
    BRCA2
  </parameter>
</function>
```

Qwen3-Coder GLM-4.5

Also common: JSON inside XML tags, plain JSON

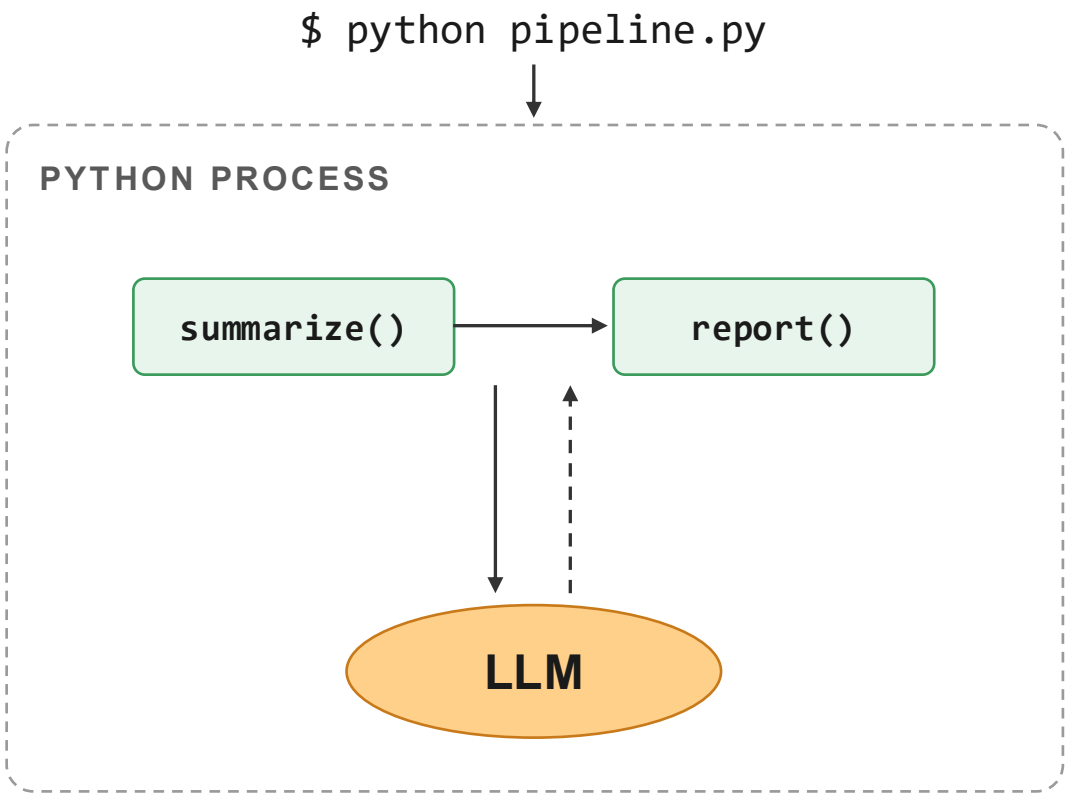
So a local model never connects to the internet?



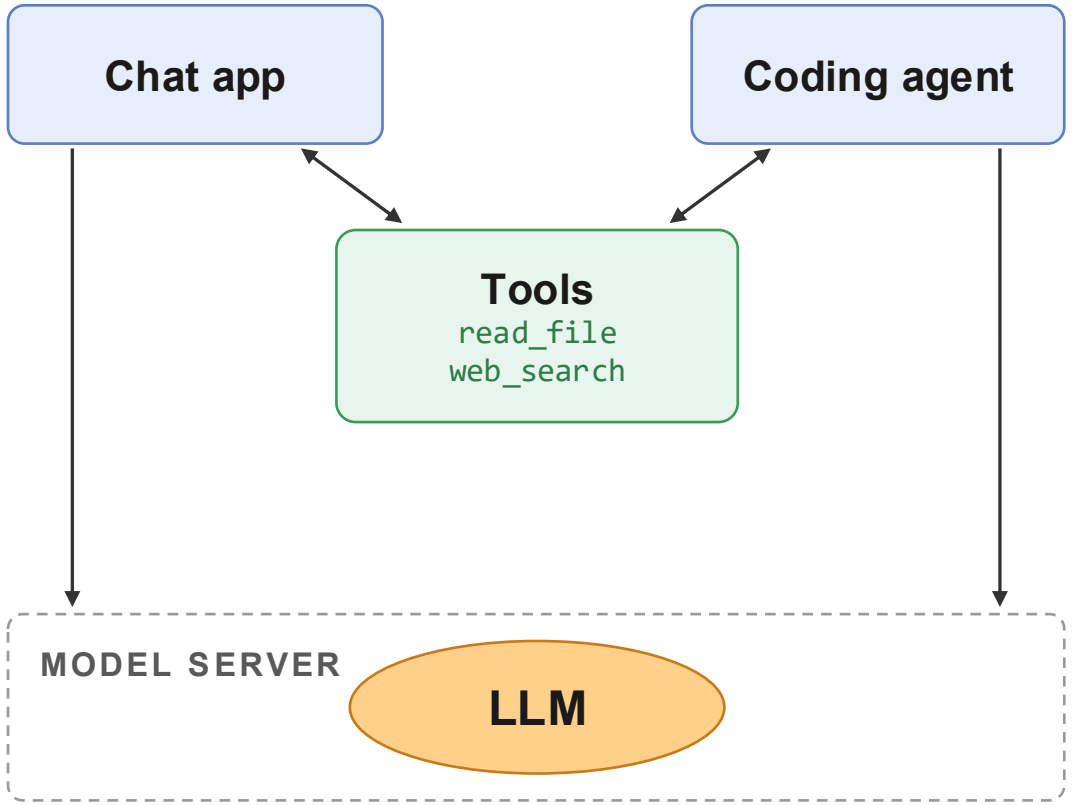
We can think of these local LLM approaches as a spectrum:

Single-purpose ⟷ General-purpose

In-process script



Model server





HARVARD MEDICAL SCHOOL | BLAVATNIK INSTITUTE BIOMEDICAL INFORMATICS

Walkthroughs

Jane Adams

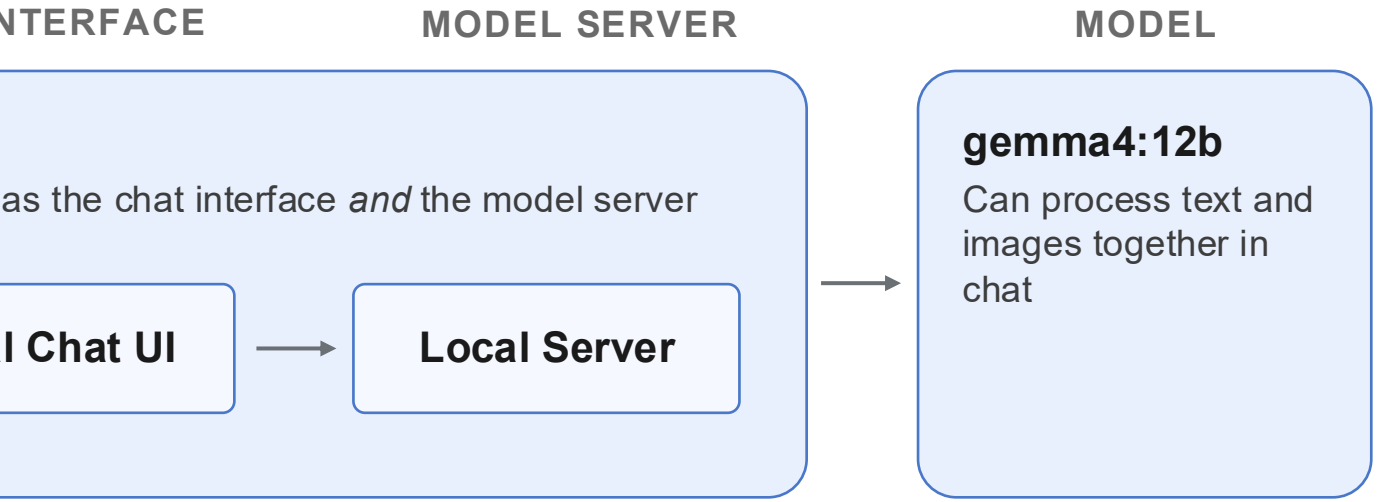
We will demo 3 ways to use LLMs locally:

	USER INTERFACE	MODEL SERVER	MODEL
1 **Ollama multimodal chat app** 	Ollama chat UI	Ollama	Gemma 4
2 opencode **Agentic coding harness**	OpenCode IDE	oMLX	DevStral 2
3 **Model server and chat from scratch**	Local web app in-browser	Python process	Phi 3.5 Instruct

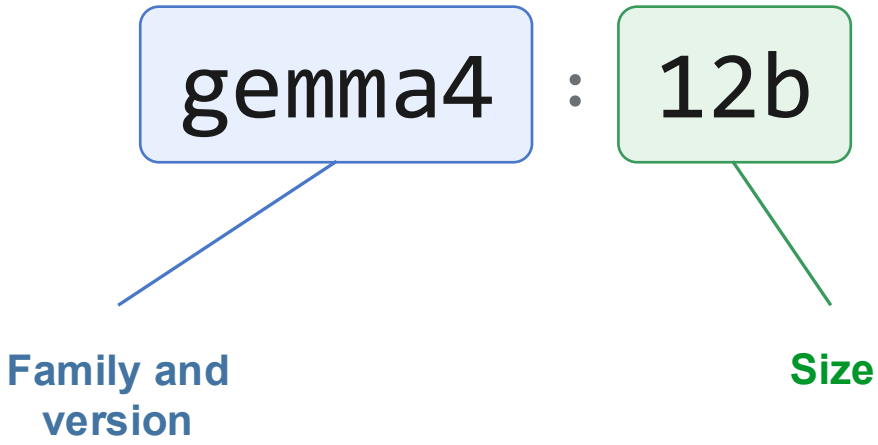
The first way we'll use LLMs locally:

1

Ollama multimodal chat app



For this Ollama multimodal chat demo we use gemma4:12b



“Multimodal” means it can process text *and* images

https://ollama.com/library/gemma4:12b

What to use local models for … and not

“Please extract the metadata in this image.”

- When was this scan taken?
- What is the orientation and zoom level?
- Is it compressed?
- Which slice are we viewing?



✓

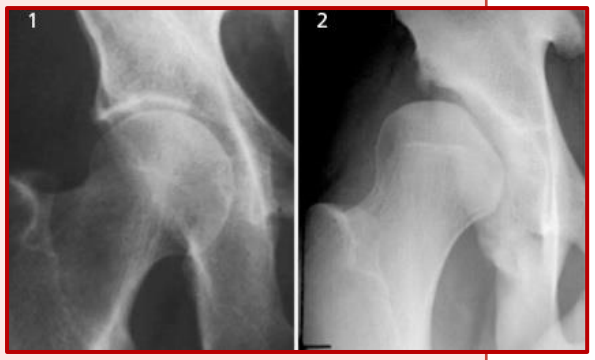
Good fit for local models

“Please diagnose and plan treatment for this patient.”

- What pathology is shown in the right image?
- How would we diagnose this patient?
- What treatment options should we pursue for this patient based on the diagnosis?

✗

Not a good fit for local models

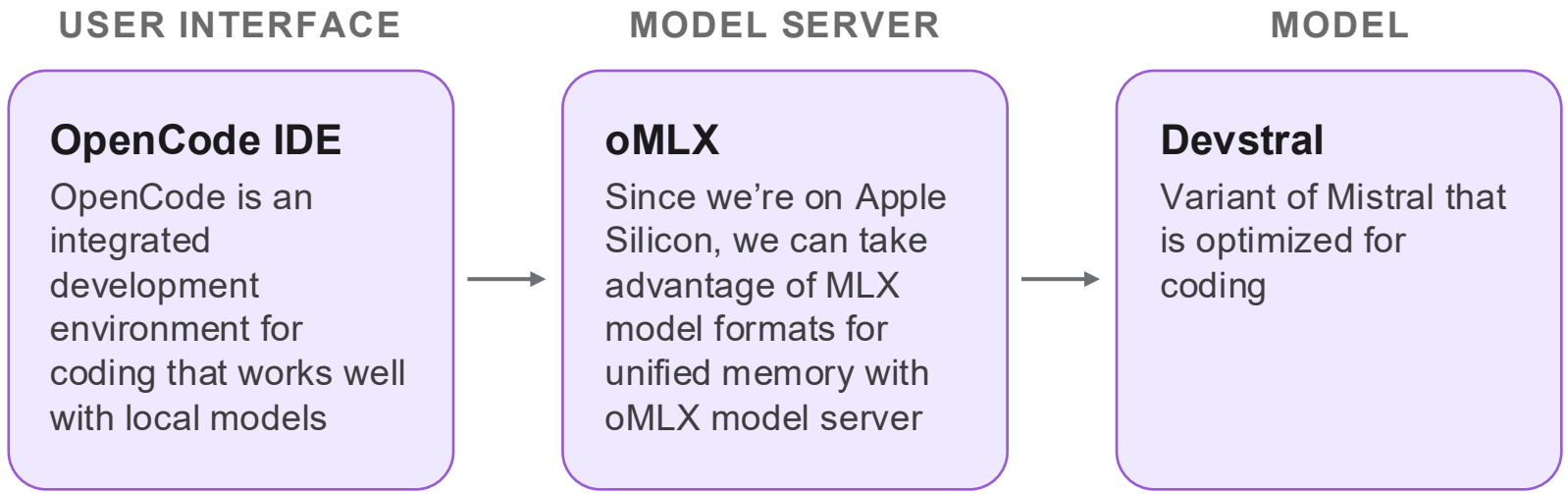


The second way we'll use LLMs locally:

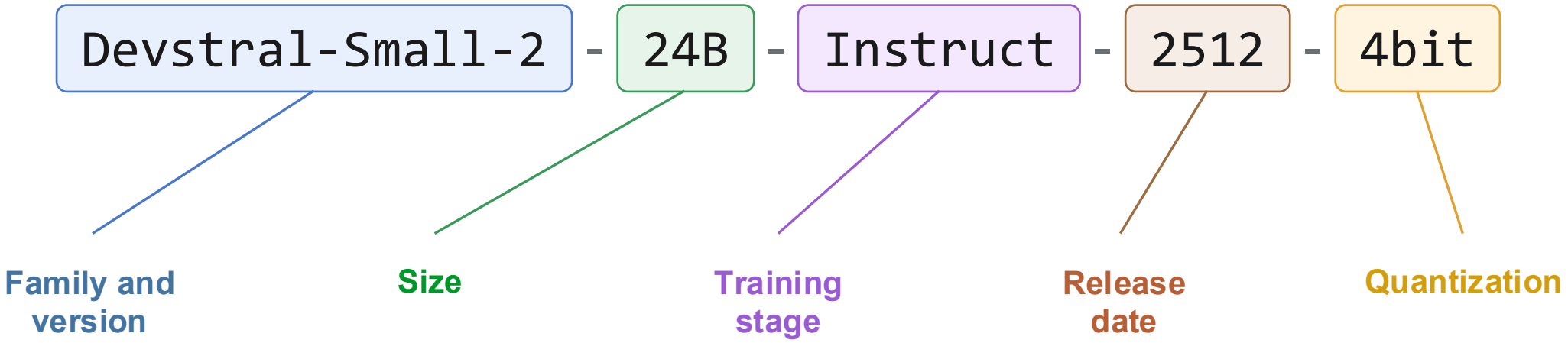
2

oMLX + opencode

Agentic coding harness with oMLX and OpenCode

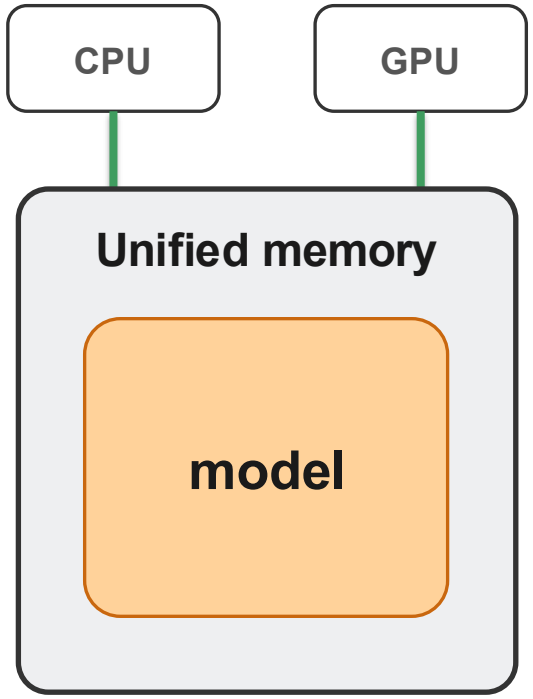


For local coding, I'm currently using:



https://huggingface.co/mistralai/Devstral-Small-2-24B-Instruct-2512

oMLX is a model server, like Ollama, but for Apple Silicon



1. Go to omlx.ai to download oMLX

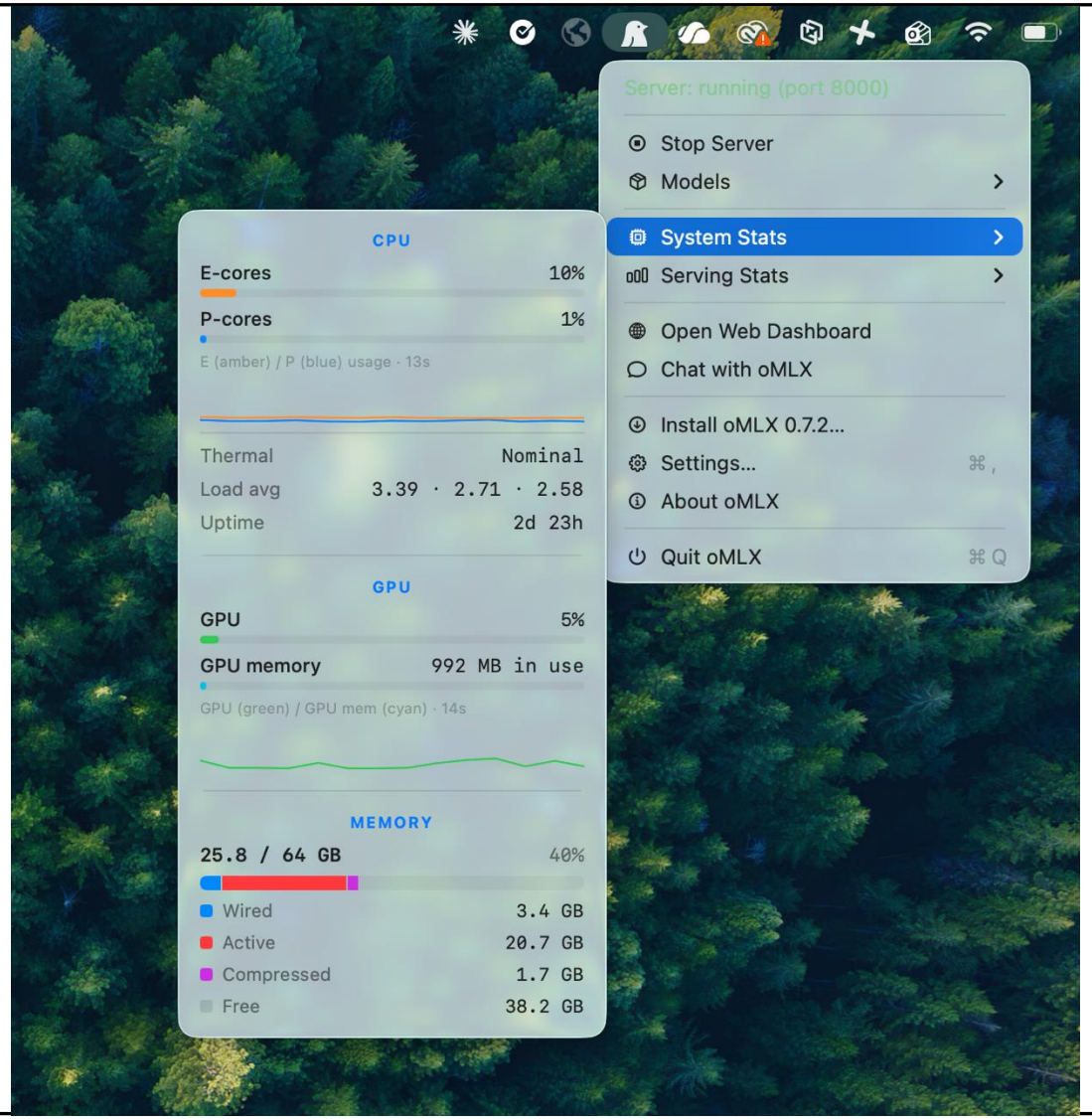
2. Walk through the installation to see how to install models

3. Go to opencode.ai to download OpenCode

4. Launch OpenCode from oMLX with `omlx launch opencode`

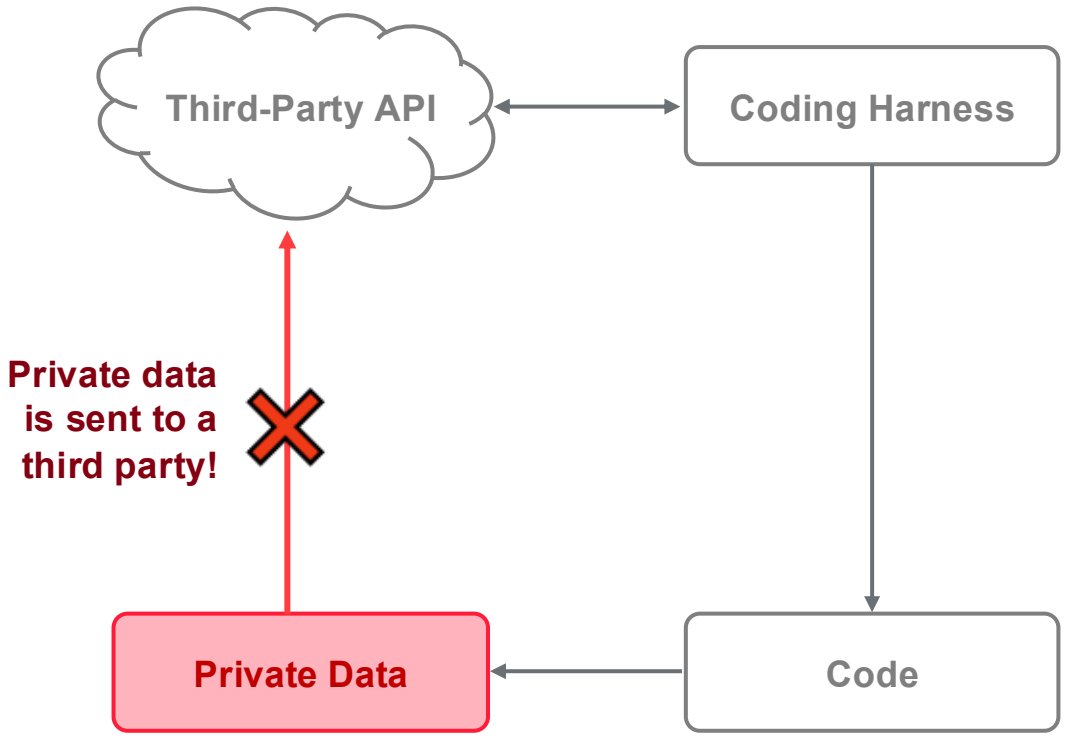
oMLX adds quick access to models and stats

System stats are available in the menu



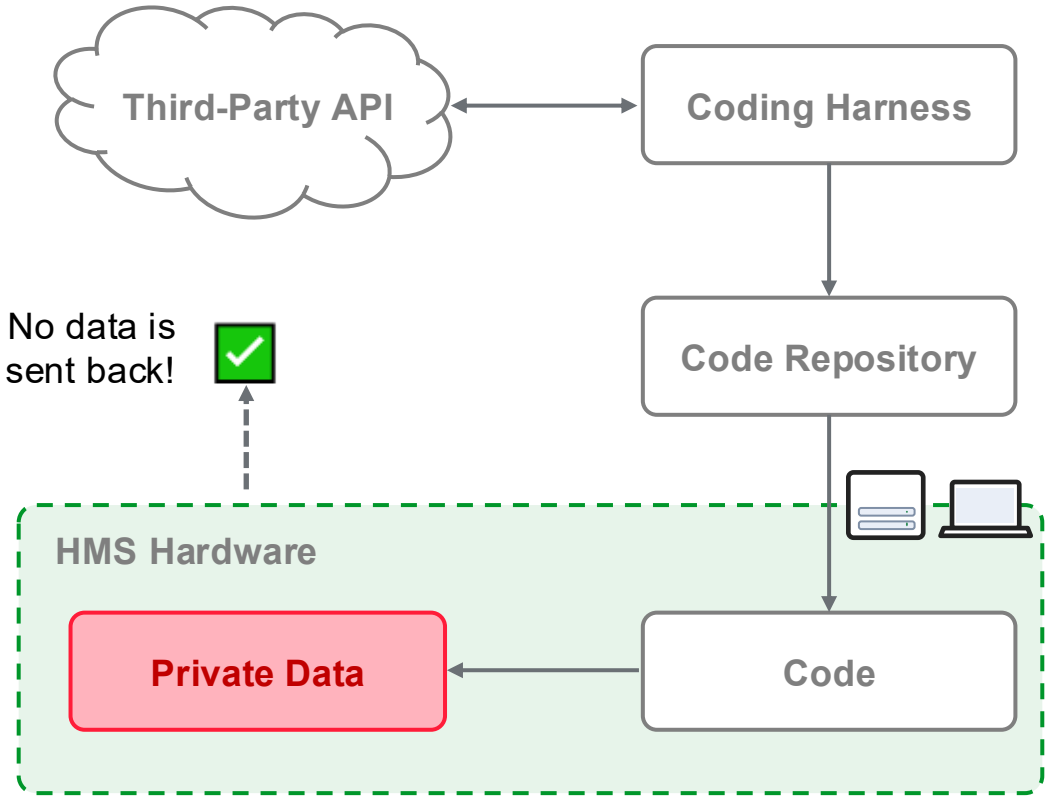
The menu bar item allows quick starting and stopping of servers

When would I actually need a local coding agent?

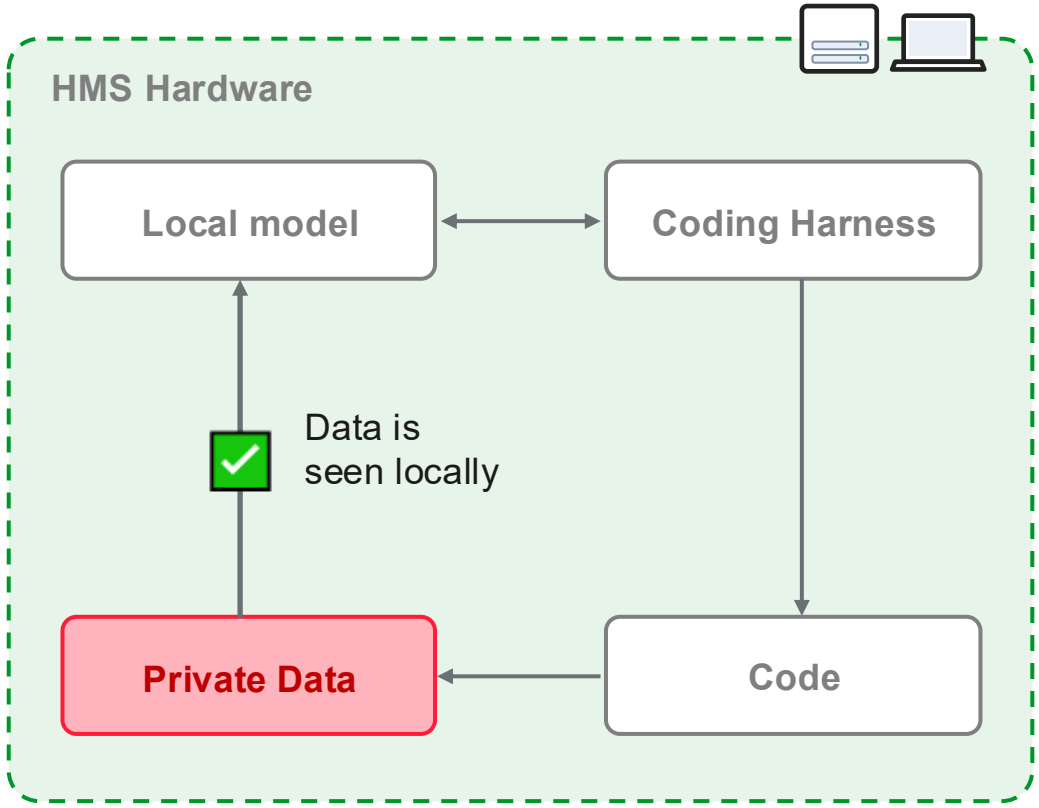


When would I actually need a local coding agent?

1 Sequester private data on HMS hardware



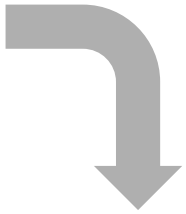
2 Model and data both stay on HMS hardware

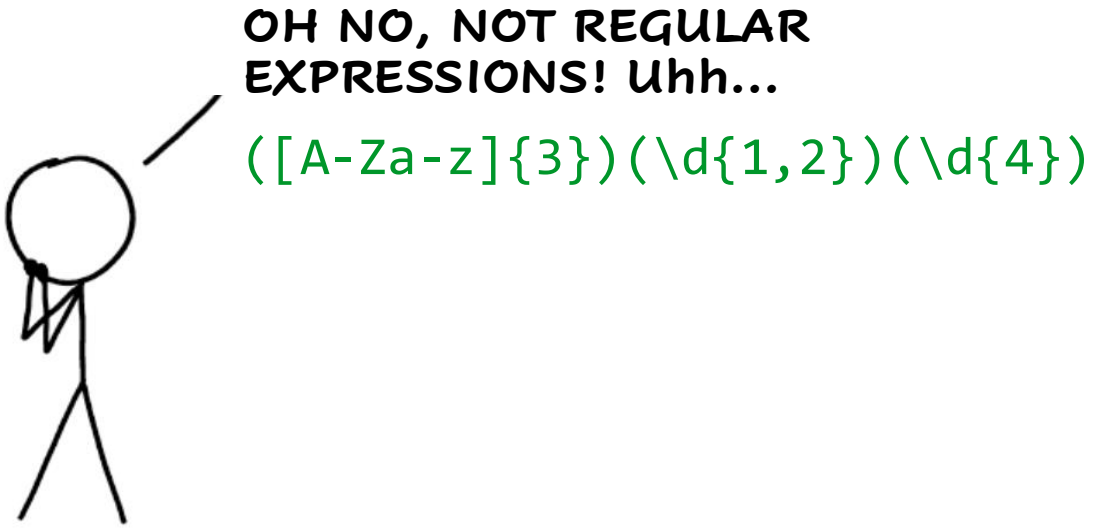


Having access to the data makes this task a lot easier:

experiments

INFO
TRTHeLaLPSDec081920482
CTRLJurkatMockOct103820235
TRTVeroHypoxiaJul291029304





experiments_parsed

Original	Treatment	Sample	CellLine	Condition	Date	PlateID
TRTHeLaLPSDec081920482	TRT	HeLa LPS	HeLa	LPS	12-08	1920482
CTRLJurkatMockOct103820235	CTRL	Jurkat Mock	Jurkat	Mock	10-10	3820235
TRTVeroHypoxiaJul291029304	TRT	Vero Hypoxia	Vero	Hypoxia	7-29	1029304

When the model can “see” the data, it can iterate:

Shell

```
cd /Users/jane/local-multimodal-example && python3 parse_samples.py
```

Executes code

```
Processed 3 experiment records.
Output written to experiments_parsed.csv
```

Read experiments_parsed.csv

Reads output (contains unpublished data!)

The samples still aren't splitting correctly. Let me analyze the patterns more carefully. The issue is that "HeLaLPS" should be split into "HeLa" and "LPS". Let me look at the actual strings more carefully:

debug_parse.py +49 -0

Writes more code based on the interim result

```
1 #!/usr/bin/env python3
2
3 def analyze_string(s):
```

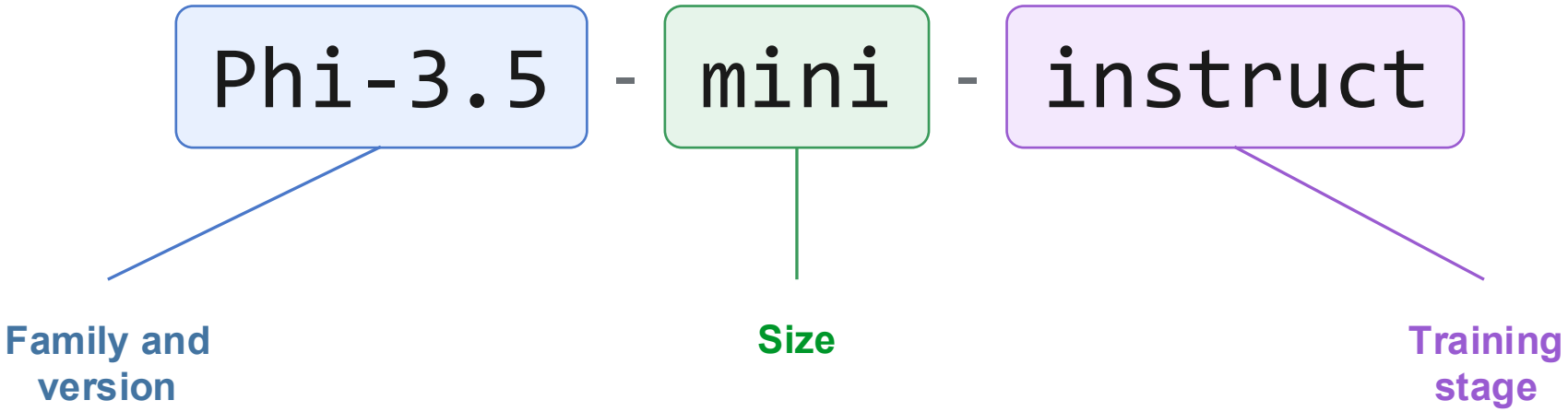
The third way we'll use LLMs locally:

3

Model server and chat from scratch

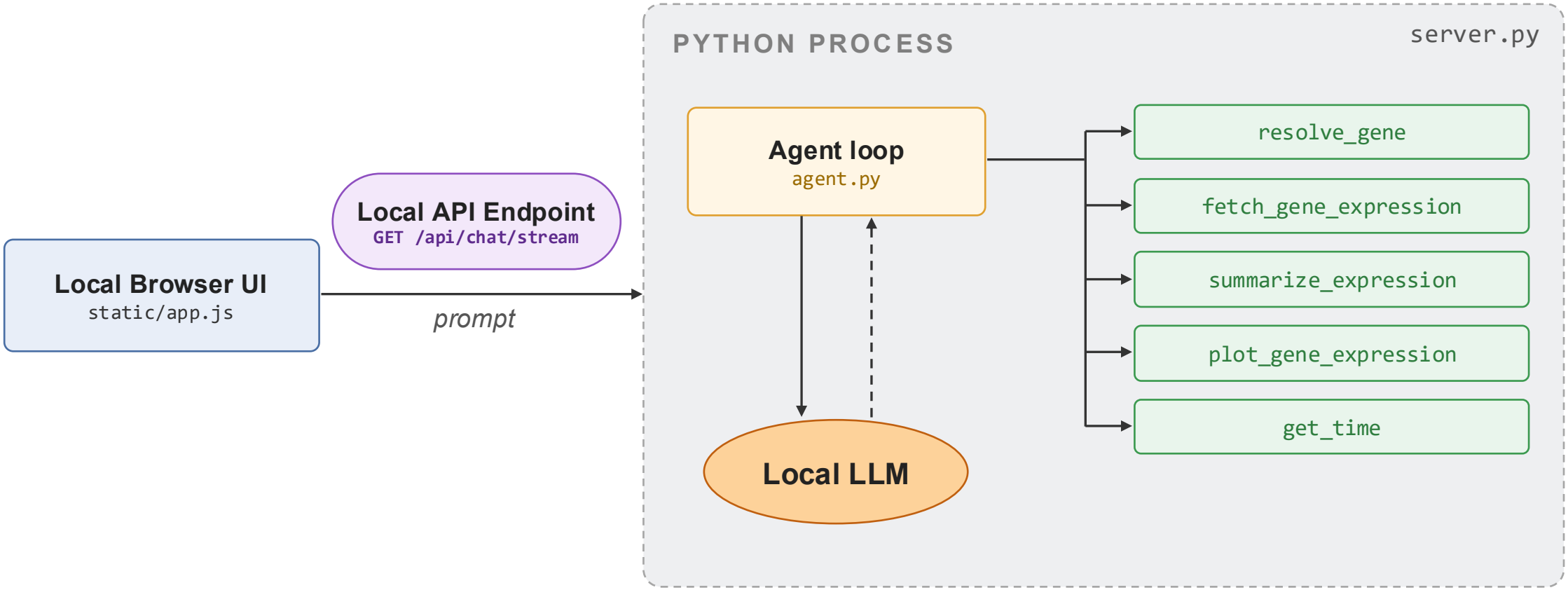
USER INTERFACE	MODEL SERVER	MODEL
Local Browser We serve a web app to function as a UI	**Local Server** We made a simple model server from scratch so you can see under the hood	**Phi-3.5 Instruct** Designed to follow instructions and call tools in-process

For this in-process demo we use Phi-3.5-mini-instruct



https://huggingface.co/microsoft/Phi-3.5-mini-instruct

We built a basic version of the model server stack from scratch:



Cloning the repo and installing a local model

1 Set up once

Requires uv

Clone the repo and install dependencies

```
$ git clone --depth 1 \
  git@github.com:ccb-hms/local-inference-example.git
$ cd local-inference-example
$ uv sync
```

2 Run a model on your laptop

Find models that fit this machine (llmfit)

```
$ uv run python scripts/find_hardware_model.py
```

Run one of the suggested models

```
$ uv run local-inference-example --model <repo-id>
```

Without `--model`, it runs the default, Phi-3.5-mini-instruct.



HARVARD MEDICAL SCHOOL | BLAVATNIK INSTITUTE BIOMEDICAL INFORMATICS

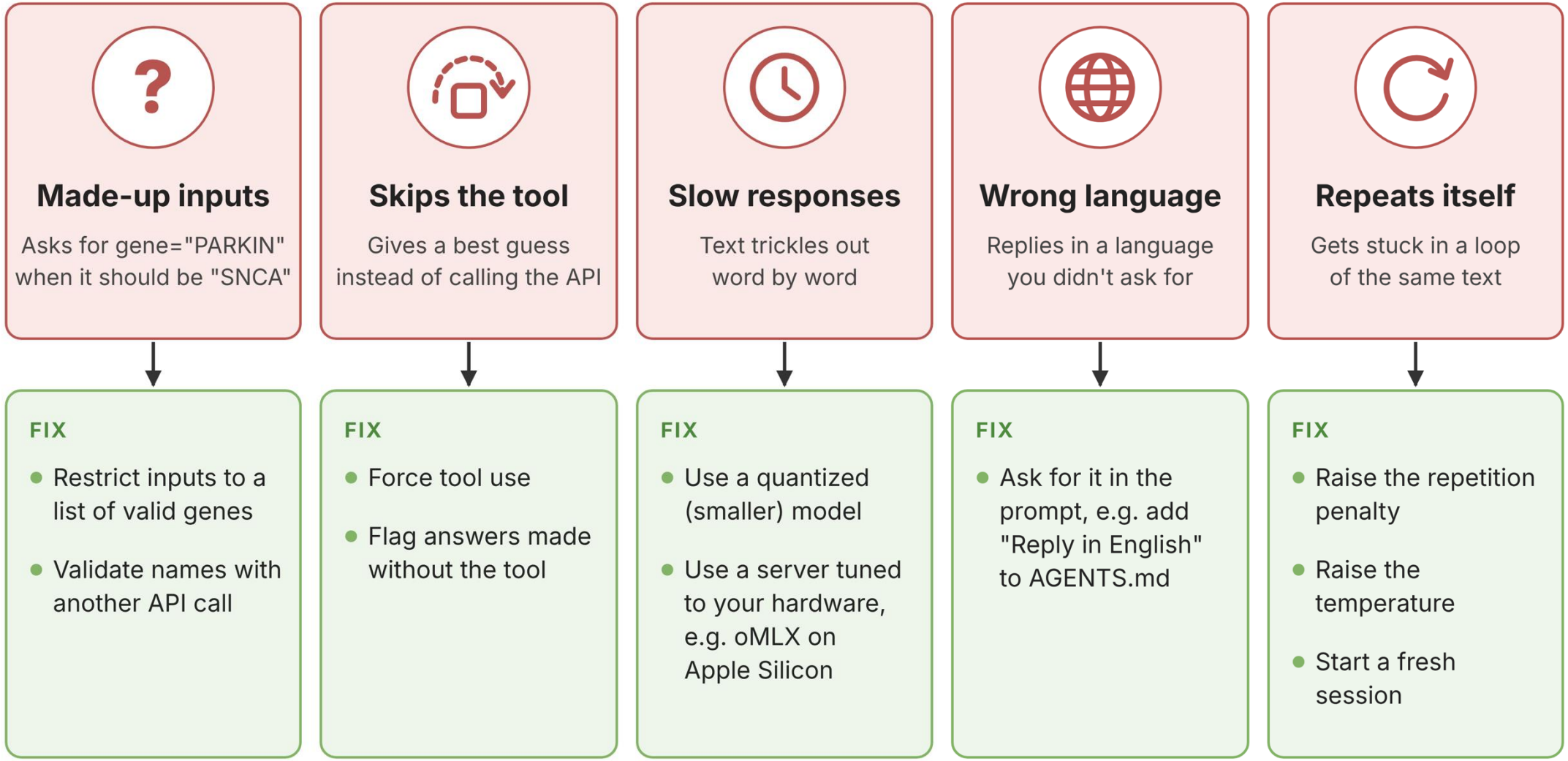
Using Models Well

Jane Adams

Customizability of local models

SETTING	TYPICAL RANGE	▲ RAISE IT WHEN	▼ LOWER IT WHEN
Temperature How random the word choice is	0.2 – 1.0 0 2	Writing is flat, repetitive, or looping	Code is sloppy or answers wander
Top-p Keeps the likeliest words until they add up to probability p	0.9 – 0.95 0 1	Output feels too predictable	You see odd or off-topic words
Top-k Keeps only the k likeliest words	20 – 64 0 100	You want more variety	You see gibberish or random tokens
Repetition penalty Penalizes words already used	1.0 (off) – 1.1 1 1.5	The model loops or repeats phrases	Code or lists get oddly reworded
Presence / frequency Penalizes reused topics and frequent words	0 – 0.5 0 2	Prose feels repetitive	Responses drift off-topic

Failure modes of local models





HARVARD MEDICAL SCHOOL | BLAVATNIK INSTITUTE BIOMEDICAL INFORMATICS

Local models at HMS

Jane Adams

HMS Data Security Levels

LEVEL 1	**Public Information:** Public and unrestricted (e.g., website posts, public event videos).
LEVEL 2	**Internal Data:** Business-sensitive, unpublished, or benign information to be held confidentially (e.g., private meeting notes).
LEVEL 3	**Confidential Information:** Sensitive information that must be safeguarded as confidential and shared only on a need-to-know basis.
LEVEL 4	**Restricted Personal Information:** Highly sensitive data including Personally Identifiable Information (PII), Protected Health Information (PHI), credentials, security secrets, or contractually restricted data.
LEVEL 5	**Extreme Risk:** Information that could be life-threatening or cause extreme institutional/national harm if disclosed.

If you are unsure, contact your Office of Research Administration https://researchadmin.hms.harvard.edu/

Where can I run larger models?

It depends on your data's security level!

O2 HPC Cluster

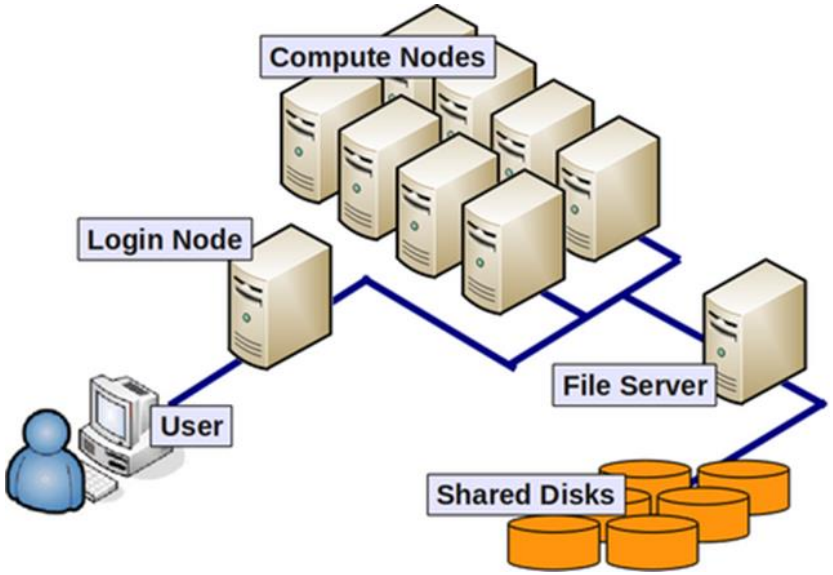
- Data security level 3
- Linux CLI
- 27 compute nodes, 133 GPUs

Longwood Cluster

- Data security level 4
- Linux CLI
- 8 compute nodes, 64 H100 GPUs

HMS Local Inference Service (coming soon)

- Data security level 4
- OpenAI and Anthropic–compatible endpoint
- GH200 server



https://harvardmed.atlassian.net/wiki/spaces/O2/overview

If you are unsure, contact your Office of Research Administration

https://researchadmin.hms.harvard.edu/



HARVARD MEDICAL SCHOOL | BLAVATNIK INSTITUTE BIOMEDICAL INFORMATICS

Beyond the Workshop

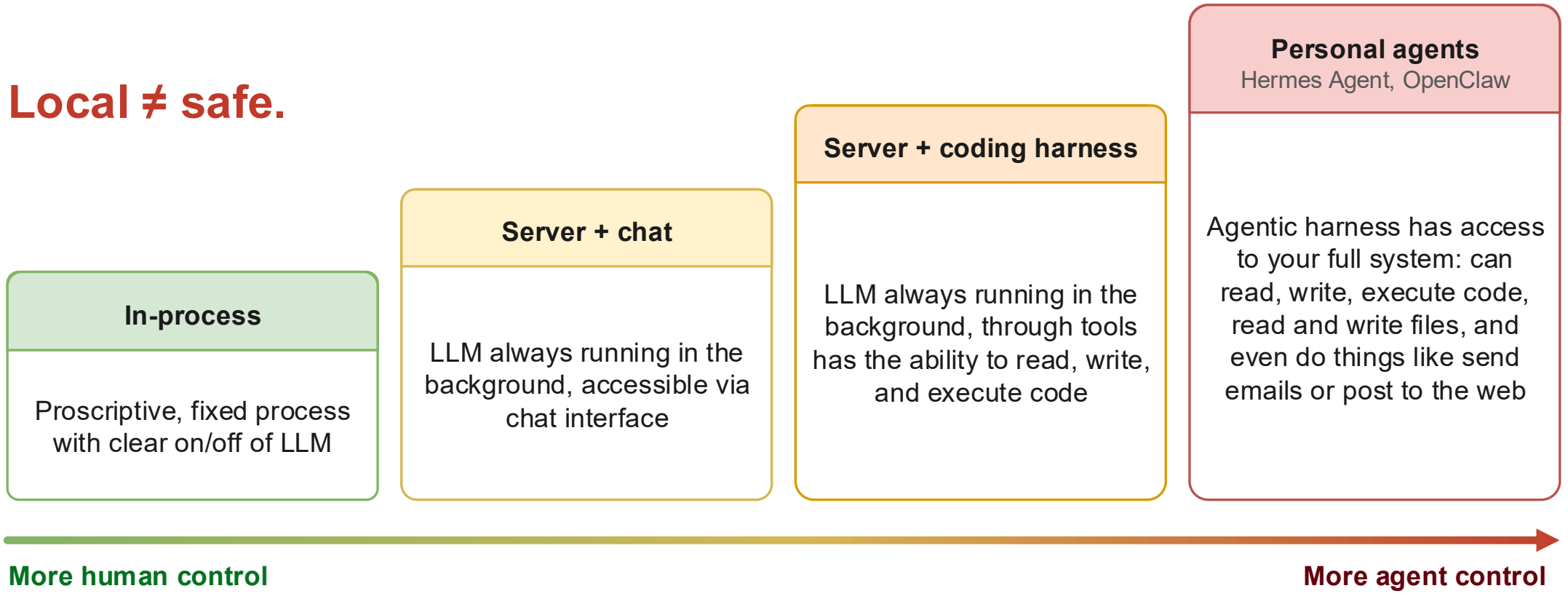
Jane Adams

What else do people do with local inference?

Each step hands the model more control over your computer…

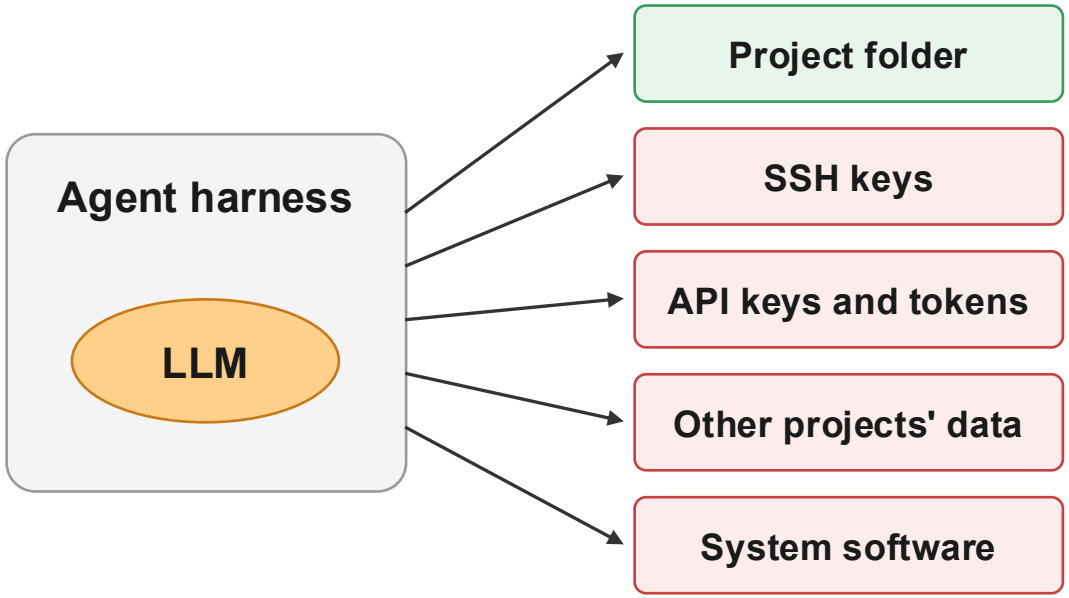
…and more risk.

Local ≠ safe.



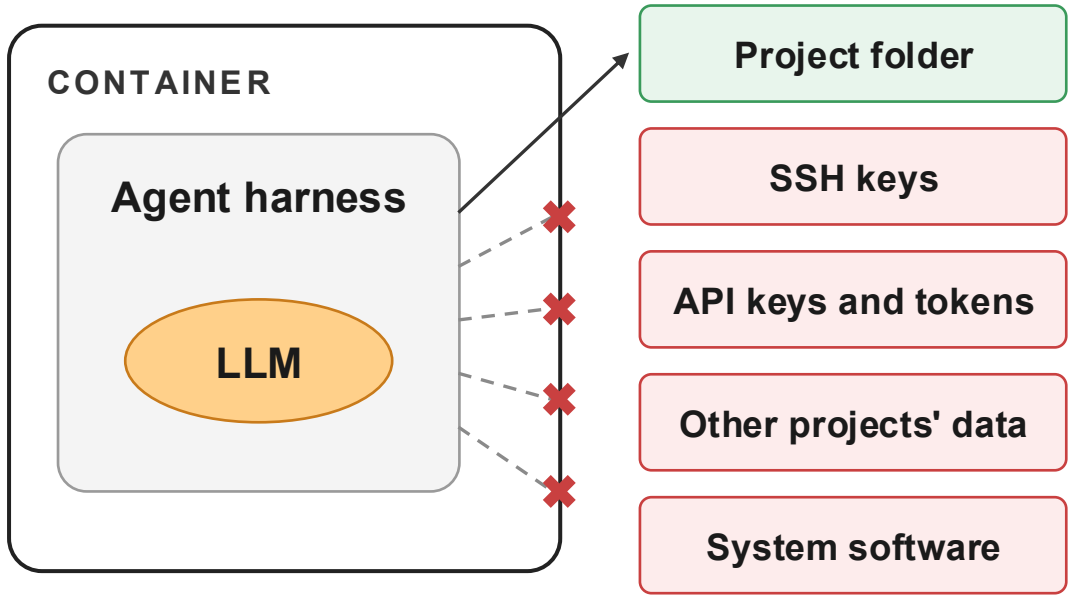
Containers add a level of safety

Without a container



The agent can read, change, or delete anything your account can reach.

With a container



The agent can use only the folders you mount into the container.

Key takeaways

1 Why local inference?

- Local keeps data inside HMS
- Make local the default, cloud as needed

2 Choosing a model

- Small specialist models often win
- Quantization can fit more parameters in smaller memory
- Prefer official uploads on model hubs

3 Talking to a model

- A tool is a function plus a description
- A harness runs the model's tool loop
- Tool-call syntax varies by model

4 Walkthrough

- Can use Ollama for local multimodal chat – but be wary of your use case!
- Consider whether you need a local coding model based on your task
- Break up coding tasks into smaller pieces
- In-process model server can be useful for understanding how things work under the hood

5 Using models well

- Tune settings before swapping models
- Restrict and validate model inputs

6 Beyond the workshop

- Local ≠ safe: agents can delete or leak
- Run agents in a container

For information about data security, contact the Office of Research Administration: https://researchadmin.hms.harvard.edu/

Come visit us!



Stuck on code, a model, a pipeline or a large data set?
Bring it to office hours.

Contact us if you need help with your project!

Office hours on Wednesdays
In person or via Zoom

Email us
ccbhelp@hms.harvard.edu

Visit our website
ccb.hms.harvard.edu

For information about data security, contact the Office of Research Administration: https://researchadmin.hms.harvard.edu/

Additional Resources

Version control / git

Python

Containers

uv package manager

3blue1brown

Udemy, Coursera

Stanford online CS courses

The missing semester of your CS education

Wizard Zines by Julia Evans

OhMyZsh!

Why not to use Jupyter Notebooks for everything

The Coding Train

Why you should stop using pip or conda and switch to uv

Pi.dev – a bare-bones coding harness that you control

https://github.com/ccb-hms/local-inference-example